Prince Sultan University

College of Computer & Information Sciences

Information Systems Department

IS499 – Senior Project

Senior Project Midterm Report (D1–D4)

Mirsad (مرصاد)

A Runtime Security Firewall for AI Agents Against Prompt-Injection Attacks

Team member	Student ID	Department
Mohammed Yasser Alanssari	221110935	IS (CYS)
Bader Alshuraie	221110016	SE
Bader Abdullah Abbod Alyamini	221110197	IS (CYS)
Abdulmalik bin Saedan	221110958	IS (CYS)
Abdullah Naif Abdullah Al-Orabi	222110432	SE (AI & DS)

Supervisor Dr. Omaia Al-Omari

Semester & Academic Year Senior Project I & II – Academic Year 2026–2027

Submission Date _____

Supervisor's Signature _____

Approval Date _____

Abstract

Large language model (LLM) agents now read e-mail, browse the web and call business tools through the Model Context Protocol (MCP) and OpenAI-style function calling. Any content an agent reads can carry hidden instructions (indirect prompt injection) that turn an authorised agent against its own organisation, and published defences are almost entirely English-centric. This project built Mirsad (مرصاد), a runtime security gateway that sits between an unmodified agent and its tools. Its objectives were to enforce action-level policy with a median overhead of at most 200 ms per call, to detect injected instructions in Arabic and English with an F1 score of at least 0.85, and to keep humans in control while reducing attack success by at least 80% and wrongly blocking no more than 5% of benign actions. Mirsad combines a declarative YAML policy engine, taint tracking of untrusted and confidential data, an Arabic/English injection detector, human approval through an installable bilingual mobile web app, and an HMAC hash-chained audit log with decision replay. On 48 benign English and Arabic tasks, full protection automatically denied 0.0% of benign calls, escalated 17.6% to a human, and 272 of 272 audited decisions replayed identically. Median added latency was 2.7 ms per call and 166 ms under 50 concurrent sessions. Detector F1 and attack success rate remain pending until the team's attack-labelled evaluation data is complete. Mirsad gives organisations an auditable control point that supports obligations under Saudi Arabia's Essential Cybersecurity Controls and Personal Data Protection Law.

الملخص

أصبح وكلاء النماذج اللغوية الكبيرة يقرؤون البريد الإلكتروني ويتصفحون الويب ويستدعون أدوات الأعمال عبر بروتوكول سياق النموذج (MCP) واستدعاء الدوال على نمط OpenAI. وقد يتضمن أي محتوى يقرؤه الوكيل تعليمات مخفية تحوّل الوكيل المصرَّح له إلى أداة ضد مؤسسته، وهو ما يُعرف بحقن الأوامر غير المباشر، في حين تكاد الدفاعات المنشورة تقتصر على اللغة الإنجليزية. طوّر هذا المشروع «مرصاد» (Mirsad)، وهو بوابة أمنية تعمل أثناء التشغيل بين الوكيل وأدواته دون الحاجة إلى تعديل الوكيل. وتمثلت أهدافه في فرض السياسات على مستوى الإجراءات على ألا يتجاوز وسيط الزمن الإضافي 200 ملّي ثانية لكل استدعاء، وكشف التعليمات المحقونة بالعربية والإنجليزية بمقياس F1 لا يقل عن 0.85، وإبقاء القرار بيد الإنسان مع خفض معدل نجاح الهجمات بنسبة 80% على الأقل، على ألا يتجاوز الحجب الخاطئ للإجراءات السليمة 5%. ويجمع مرصاد بين محرك سياسات تصريحي بصيغة YAML، وتتبّع مصدر البيانات غير الموثوقة والسرية، وكاشف حقن للعربية والإنجليزية، وموافقة بشرية عبر تطبيق ويب ثنائي اللغة قابل للتثبيت على الهاتف، وسجل تدقيق مترابط بسلسلة تجزئة HMAC يتيح إعادة تشغيل القرارات. وفي 48 مهمة سليمة بالعربية والإنجليزية، بلغت نسبة الاستدعاءات السليمة التي رفضتها الحماية الكاملة آليًا 0.0%، وأحالت 17.6% منها إلى مراجع بشري، وطابقت إعادةُ التشغيل القرارَ الأصلي في 272 من أصل 272 قرارًا مسجّلًا. وبلغ وسيط الزمن الإضافي 2.7 ملّي ثانية لكل استدعاء، و166 ملّي ثانية مع 50 جلسة متزامنة. أما مقياس F1 للكاشف ومعدل نجاح الهجمات فما زالا معلّقين إلى حين اكتمال بيانات الهجمات المصنّفة. ويقدّم مرصاد للمؤسسات نقطة تحكم قابلة للتدقيق تدعم وفاءها بضوابط الهيئة الوطنية للأمن السيبراني ونظام حماية البيانات الشخصية.

Table of Contents

Abstract i
الملخص ii
Table of Contents iii
List of Figures vi
List of Tables viii
List of Acronyms x
Chapter 1 – Introduction 1
1.1 Background 1
1.2 Problem Statement 2
1.3 Project Objectives 2
1.4 Scope and Constraints 3
1.5 Stakeholders and User Personas 4
1.5.1 Stakeholder Personas 4
1.6 Industry Relevance & IS Application 5
1.7 Project Timeline (Gantt Chart) 6
Chapter 2 – Literature Review 8
2.1 Overview of Previous Studies 8
2.1.1 LLM agents and tool use 8
2.1.2 Prompt injection: origins and taxonomy 9
2.1.3 Benchmarks for agent security 11
2.1.4 Defences at the model, prompt, runtime and architecture levels 12
2.1.5 Multilingual and Arabic LLM safety 14
2.1.6 Standards and Saudi regulation 15
2.1.7 Foundations used by Mirsad 16
2.2 Comparative Analysis 17
2.3 Research Gap and Positioning 21
Chapter 3 – Methodology 23
3.1 Research Design / Project Approach 23
3.2 Data Collection / Requirements Gathering 24

3.2.1 Elicitation methods 24
3.2.2 Threat model and assumptions 25
3.2.3 Use-case model 25
3.2.4 Requirements specification 29
3.3 System Design & Architecture 32
3.3.1 Architectural style and system context 33
3.3.2 Components and responsibilities 35
3.3.3 Core processes 39
3.3.4 Policy language design 44
3.3.5 Taint-tracking design 47
3.3.6 Injection detector design 49
3.3.7 Database design 50
3.3.8 Security design and threat model 53
3.3.9 Interface design 55
3.3.10 User-interface design 57
3.3.11 Deployment architecture and technology stack 61
3.4 Implementation Strategy 63
3.4.1 Development methodology 63
3.4.2 Mapping to the IS499 deliverables 64
3.4.3 Team roles and responsibilities 65
3.4.4 Development environment, tools and version control 65
3.4.5 Changes from the proposal and their justification 66
3.4.6 Budget 67
3.6 Ethical, Legal & Security Considerations 67
3.6.1 Data privacy and the PDPL 67
3.6.2 Regulatory and standards alignment 68
3.6.3 Security of the gateway itself 68
3.6.4 Responsible handling of attack material 69
3.6.5 Social and professional considerations 69
3.6.6 Licences, third-party components and AI assistance 69
References 71

List of Figures

Figure 1.1: Project schedule with task owners (abbreviations as in the text) and milestones M1–M7 7

Figure 2.1: A three-axis taxonomy of prompt-injection attacks: the channel that carries the text, the technique used to obtain compliance and the attacker's goal (synthesised from the sources reviewed in Section 2.1.2)..... 10

Figure 3.1: Use-case diagram with misuse cases (black) and the security use cases that mitigate them 26

Figure 3.2: System context: Mirsad sits between the AI agent and the enterprise tools; the attacker influences only the content stored in those tools..... 33

Figure 3.3: Mirsad architecture: protocol adapters, the call path (policy engine, rate limiter, taint tracker, combining step, human approval), the result path (injection detector, sanitising and spotlighting, taint labelling) and the shared services..... 34

Figure 3.4: Component (package) diagram. Dashed "injected" arrows are runtime collaborators passed to the gateway's constructor; every package also uses the foundation modules..... 36

Figure 3.5: Class diagram of the gateway, its collaborating services and the shared data model (signatures simplified; the full interfaces are in the engineering contract)..... 38

Figure 3.6: Sequence of a tool call through the gateway: decision, execution and inspection of the result..... 40

Figure 3.7: Decision logic for a tool call. Constraints, rate limits and global rules all report their reasons before the combining step..... 41

Figure 3.8: Sequence of an escalated call: approval request, notification, human decision and resumption of the waiting call..... 42

Figure 3.9: State machine of an approval request; every terminal state other than Approved blocks the call 43

Figure 3.10: Context-level (level 0) data-flow diagram of Mirsad..... 44

Figure 3.11: Level 1 data-flow diagram: processes 1.0 to 6.0 and data stores D1 to D4 . 44

Figure 3.12: Entity-relationship diagram of the six tables. The solid line is the only enforced foreign key; dashed lines are logical references by value..... 51

Figure 3.13: Threat model with trust boundaries A to E and the STRIDE categories of each data flow 53

Figure 3.14: Activity and audit page with the detail drawer of an approval decision: summary, previous hash and event hash, and the raw event 59

Figure 3.15: Policy editor in Arabic: the sidebar and controls are mirrored, while the YAML policy stays left-to-right 59

Figure 3.16: Approver PWA in Arabic: pending requests with risk label, countdown and plain-language summary 60

Figure 3.17: Approver PWA request detail in English: risk, countdown, explanation sections and the Approve and Deny buttons 61

Figure 3.18: Deployment architecture: the Mirsad container, the database, optional components and the clients 62

List of Tables

Table 1: Acronyms used in this report x

Table 1.1: Project scope 3

Table 1.2: Stakeholders and their interests 4

Table 1.3: Project phases, durations and responsible members (from the D2 work breakdown) 6

Table 1.4: Project milestones 6

Table 2.1: Key studies on agent-security evaluation and their implications for Mirsad .. 11

Table 2.2: Feature comparison of runtime guardrails, MCP gateways, research systems and Mirsad (N/D = not documented in the sources reviewed) 18

Table 3.1: Use-case catalogue 26

Table 3.2: Use-case specification UC-01 – Invoke tool via gateway 27

Table 3.3: Use-case specification UC-05 – Approve / deny action 28

Table 3.4: Use-case specification UC-08 – Manage policies 28

Table 3.5: Use-case specification UC-14 – Verify audit integrity 28

Table 3.6: Functional requirements 29

Table 3.7: Non-functional requirements 31

Table 3.8: Main design decisions 35

Table 3.9: Components, responsibilities and interfaces 36

Table 3.10: Structure of a policy document (schema version 1) 45

Table 3.11: Reason codes of the policy, detector, approval and system layers 46

Table 3.12: Taint rules evaluated for each tool call 48

Table 3.13: Data dictionary of the database tables 52

Table 3.14: STRIDE analysis of the gateway 53

Table 3.15: Agent-facing interfaces 56

Table 3.16: REST API overview 56

Table 3.17: Sprint plan 63

Table 3.18: IS499 deliverables and where they are addressed 64

Table 3.19: Workstream ownership 65

Table 3.20: Technology stack and justification 65

Table 3.21: Original and revised budget (SAR) 67

List of Acronyms

Table 1: Acronyms used in this report

Acronym	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
ASR	Attack Success Rate
ATLAS	Adversarial Threat Landscape for Artificial-Intelligence Systems (MITRE)
AUC	Area Under the (ROC) Curve
CEL	Common Expression Language
CI	Continuous Integration
CISO	Chief Information Security Officer
CLI	Command-Line Interface
CORS	Cross-Origin Resource Sharing
CPU / GPU	Central / Graphics Processing Unit
CRM	Customer Relationship Management
CSP	Content Security Policy
CSV	Comma-Separated Values
DFD	Data-Flow Diagram
ECC	Essential Cybersecurity Controls (NCA)
ERD	Entity-Relationship Diagram
F1	Harmonic mean of precision and recall
FPR	False-Positive Rate
FR / NFR	Functional / Non-Functional Requirement
HMAC	Hash-based Message Authentication Code
HS256	HMAC with SHA-256 (JSON Web Token signature algorithm)
HTML	HyperText Markup Language
HTTP(S)	Hypertext Transfer Protocol (Secure)
IBAN	International Bank Account Number
IP	Internet Protocol
ISO / IEC	International Organization for Standardization / International Electrotechnical Commission
JSON	JavaScript Object Notation
JWT	JSON Web Token
KLM	Keystroke-Level Model

Acronym	Meaning
LLM	Large Language Model
MCP	Model Context Protocol
MoSCoW	Must, Should, Could, Won't prioritisation
MSA	Modern Standard Arabic
NCA	National Cybersecurity Authority (Saudi Arabia)
NFKC	Unicode Normalization Form KC (compatibility composition)
NIST	National Institute of Standards and Technology
OPA	Open Policy Agent
ORM	Object-Relational Mapping
OWASP	Open Worldwide Application Security Project
PDPL	Personal Data Protection Law (Saudi Arabia)
PEP	Policy Enforcement Point
PWA	Progressive Web App
RBAC	Role-Based Access Control
ReDoS	Regular-Expression Denial of Service
REST	Representational State Transfer
RMF	Risk Management Framework (NIST AI RMF)
ROC	Receiver Operating Characteristic
RTL	Right-to-Left
SAR	Saudi Riyal
SAST	Static Application Security Testing
SDAIA	Saudi Data and AI Authority
SDK	Software Development Kit
SHA	Secure Hash Algorithm
SIEM	Security Information and Event Management
SMART	Specific, Measurable, Achievable, Relevant, Time-bound
SPAP	Senior Project Agreement and Proposal
SQL	Structured Query Language
SRS	Software Requirements Specification
SSE	Server-Sent Events
SSRF	Server-Side Request Forgery
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
TLS	Transport Layer Security
UAT	User Acceptance Testing
UC	Use Case

Acronym	Meaning
UI / UX	User Interface / User Experience
URL	Uniform Resource Locator
VAPID	Voluntary Application Server Identification (Web Push)
VAT	Value-Added Tax
VM	Virtual Machine
WCAG	Web Content Accessibility Guidelines
XACML	eXtensible Access Control Markup Language
YAML	YAML Ain't Markup Language

Chapter 1 – Introduction

1.1 Background

Large language models (LLMs) have moved quickly from conversational assistants to *agents*: software that receives a goal in natural language, plans a sequence of steps, and carries those steps out by calling tools. A modern agent can read a mailbox, search a customer relationship management (CRM) system, open web pages and documents, call internal APIs, send messages and even initiate payments. Two technical developments made this practical. The first is native *function calling*, in which the model emits a structured request (a tool name plus JSON arguments) that the host application executes and whose result is fed back to the model, following the reason-and-act loop popularised by ReAct [1] and tool-learning work such as Toolformer [2]. The second is the Model Context Protocol (MCP) [3], an open protocol that standardises how agents discover and invoke tools hosted by independent servers, so the same agent can be connected to many enterprise systems without bespoke integration code.

This capability creates a new security problem. An agent treats everything placed in its context window as potential instructions, including the *content returned by its tools*. If an attacker can influence any of that content, for example by sending an e-mail to the organisation, editing a public web page, uploading a CV, or writing a product review, the attacker can embed instructions that the agent may follow as if they came from its user. This class of attack, *indirect prompt injection*, was first demonstrated against real LLM-integrated applications by Greshake et al. [4] and is ranked first (LLM01) in the OWASP Top 10 for LLM Applications [5]. Standardised benchmarks such as AgentDojo [6] and InjecAgent [7] have since shown that tool-using agents built on leading models remain vulnerable, and that the risk is greatest when three ingredients are present together: access to private data, exposure to untrusted content, and the ability to communicate externally [8].

The Kingdom of Saudi Arabia is investing heavily in artificial intelligence under Vision 2030 [9], with the Saudi Data and AI Authority (SDAIA) issuing AI ethics principles [10] and the National Cybersecurity Authority (NCA) mandating the Essential Cybersecurity Controls (ECC) for national entities [11]. Banks, government service providers and customer-support centres in the Kingdom serve their customers largely in Arabic, so the agents they adopt will need to read and write Arabic. Two facts make this context distinctive. First, the Personal Data Protection Law (PDPL) [12] makes unauthorised disclosure of personal data, exactly the outcome of a successful exfiltration attack, a regulatory violation. Second, almost all published prompt-injection defences and benchmarks are English-centric, while research on multilingual jailbreaks shows that

safety measures trained mainly on English degrade on other languages, scripts and transliterations [13], [14], [15].

Mirsad (Arabic مرصاد, "watchtower" or "observation post") is our response: a runtime security gateway that sits between an agent and its tools, inspects every tool call and every tool result, and enforces the organisation's security policy at the point where the agent acts on the world.

1.2 Problem Statement

Organisations that connect LLM agents to business systems currently have no practical, language-aware control point that can stop a hijacked agent from *acting* on injected instructions. The problem has four parts.

1. **Conventional controls cannot see the attack.** The malicious instruction travels as ordinary data (an e-mail body, a web page) and the harmful action is issued by an authorised agent using valid credentials. Web application firewalls, identity and access management and data-loss prevention tools observe legitimate traffic and therefore do not intervene.
2. **Most defences classify text instead of governing actions.** Prompt-injection classifiers and guardrail frameworks operate at the model boundary and produce a probability that a text is malicious. A classifier that misses a novel phrasing provides no further protection, and none of these tools expresses rules such as "this agent may read the CRM but may never export more than 50 records" or "never e-mail data that came from the CRM to an external domain".
3. **Architectural defences are hard to adopt.** Research designs that separate trusted control flow from untrusted data, such as CaMeL [16] and the design patterns of Beurer-Kellner et al. [17], are strong but require the agent to be rebuilt around a specific architecture, which is unrealistic for organisations that buy or reuse agents.
4. **Arabic is largely untested.** Detectors and benchmarks are trained and evaluated almost exclusively on English, leaving Modern Standard Arabic, Gulf dialect, Arabizi and mixed Arabic-English injections without measured protection.

The project therefore asks: *can a drop-in gateway, placed at the standard tool-calling boundary and requiring no change to the agent, measurably reduce the success of indirect prompt-injection attacks in Arabic and English while keeping legitimate work flowing, adding little latency, and leaving an auditable record of every decision?*

1.3 Project Objectives

The proposal (SPAP form) listed six objectives, O1–O6. Following the report template, they are consolidated below into three SMART objectives; the original objectives each one covers are shown in brackets, and the measurable targets are the success criteria agreed in the proposal.

1. **Enforce action-level security at the tool boundary (O1, O2, O3).** By milestone M5 (March 2027), deliver a model-agnostic gateway that intercepts 100% of the tool calls and tool results of agents that use MCP (stdio and HTTP) or OpenAI-style function calling. The gateway enforces a declarative YAML policy with per-tool and per-parameter constraints and rate limits, uses taint tracking to block or escalate flows of untrusted or confidential data to external destinations, and adds a median latency of no more than 200 ms per tool call.
2. **Detect injected instructions in Arabic and English (O4).** By milestone M4 (February 2027), build a bilingual (Modern Standard Arabic, Gulf dialect, Arabizi, mixed Arabic-English and English) prompt-injection detector, together with a documented, labelled dataset, that reaches an F1 score of at least 0.85 on held-out Arabic and English test sets that were written separately from the training data.
3. **Keep humans in control and prove the effect (O5, O6).** By milestone M6 (April 2027), provide a mobile approval application and a web dashboard with a tamper-evident audit log in which every blocked or approved action can be replayed. On an Arabic-extended agent benchmark (Mirsad-Bench), show that the gateway reduces attack success by at least 80% relative to an unprotected baseline while wrongly blocking no more than 5% of benign actions.

1.4 Scope and Constraints

Table 1.1 summarises what is in and out of scope.

Table 1.1: Project scope

	Items
In scope	Gateway core and adapters (MCP proxy, OpenAI-compatible proxy, Guard API and Python SDK); declarative policy language and engine; taint tracking; Arabic/English injection detector and its dataset; human-approval workflow with an installable mobile web application; administration dashboard; hash-chained audit log with verification and replay; a simulated enterprise workspace and demonstration agent; the Mirsad-Bench benchmark; documentation and an open-source release.
Out of scope	Protecting the LLM itself from direct jailbreaks typed by its own user; network- and host-level security; securing model training data or weights; production-grade high availability, horizontal scaling and multi-tenant isolation; integration with real customer data.

Technical constraints. The project is developed by five students on personal laptops without GPUs. The demonstration agent and an LLM smoke run therefore use an open-weight model (Qwen2.5-7B) run locally through Ollama, the main benchmark uses a deterministic scripted agent, and the detector must run on a CPU within the latency budget. MCP is a young, evolving specification, so protocol handling is isolated behind an adapter layer. The demonstration environment uses simulated data only.

Organisational constraints. Work is limited to two academic semesters with bi-weekly supervisor meetings and a modest budget (Section 3.4.6 and Appendix A.2). Each workstream has a single owner, so cross-training and code review are needed to avoid single points of failure.

Ethical and legal constraints. No real personal data is used; all customers, e-mails and accounts in the demonstration workspace and benchmark are fictional and use reserved `.example` domains. Attack payloads are used only against the project's own simulated environment. The gateway must not become a privacy risk itself, so secrets are never logged and personal identifiers in the audit log are masked in line with the PDPL [12]. AI-assisted work is disclosed (Section 3.6).

1.5 Stakeholders and User Personas

Table 1.2 lists the stakeholders, their interests and how Mirsad serves them.

Table 1.2: Stakeholders and their interests

Stakeholder	Interest in Mirsad	How Mirsad serves them
Chief Information Security Officer (CISO) / security team	Allow AI agents without creating an uncontrolled data-exfiltration channel; evidence for NCA ECC compliance	Central policies, blocked-attack statistics, tamper-evident audit evidence
Approvers (line managers, security analysts)	Decide quickly on risky actions without reading logs	Push alert with a plain-language Arabic/English summary and one-tap approve/deny
Agent developers / integrators	Secure an existing agent without rewriting it	Drop-in MCP proxy, OpenAI-compatible endpoint, Python SDK
Business users (employees using the agent)	Agent keeps working for legitimate tasks	Low false-positive rate; only high-risk actions are paused
Compliance officers / internal auditors	Reconstruct who allowed what and why	Searchable, exportable, verifiable audit log with decision replay
Data protection officer	PDPL compliance, no leakage of personal data	Taint rules for confidential data, masking of identifiers in logs
IT operations	Simple deployment and low overhead	Single-service deployment (one gateway container plus a database), health endpoint, median added latency target of at most 200 ms
Customers of the organisation (indirect)	Their personal data is not leaked or misused	Exfiltration and destructive actions blocked or escalated

1.5.1 Stakeholder Personas

Persona 1 – Noura Al-Qahtani, Security Operations Analyst (primary user of the approval app). Noura, 29, works in the security operations centre of a Riyadh bank that has deployed an AI assistant for its customer-support team. She is comfortable with

security tools but cannot read every log line and is often away from her desk. She needs to know immediately when the assistant attempts something risky, to understand in a few seconds *what* the agent wants to do and *why* it was paused, and to decide from her phone. She reads Arabic and English and prefers Arabic on mobile. *Goals:* fast, confident decisions; no surprises in the audit. *Frustrations:* alerts without context; false alarms. *Mirsad features:* push notification, bilingual plain-language summary, "where the data came from" section, decision in under 30 seconds.

Persona 2 – Faisal Al-Harbi, AI Platform Engineer (agent developer). Faisal, 33, integrates LLM agents with the bank's CRM and e-mail through MCP servers and does not want to rewrite agents each time security requirements change. *Goals:* put a control point in front of existing agents by changing a URL; express rules as configuration; test policies safely. *Frustrations:* security reviews that block releases; tools that only work with one LLM vendor. *Mirsad features:* MCP and OpenAI-compatible proxies, YAML policies with validation and monitor mode, detector playground.

Persona 3 – Huda Al-Shehri, Compliance and Internal Audit Officer. Huda, 41, prepares evidence for NCA ECC and PDPL audits. *Goals:* prove that controls work, reconstruct incidents, export evidence. *Frustrations:* logs that can be edited silently; decisions that cannot be explained. *Mirsad features:* HMAC hash-chained audit log, integrity verification, decision replay, CSV/JSON export.

1.6 Industry Relevance & IS Application

Mirsad is an information-systems solution to an organisational problem as much as a technical one. It places a *policy enforcement point* in the information flow between a new class of actor (the AI agent) and existing business systems, so that the organisation's rules, rather than the model's judgement, decide what data may leave and which actions need a human. This mirrors established IS governance practice: segregation of duties (the agent proposes, a human approves high-risk actions), least privilege (per-tool and per-parameter permissions), and accountability (a verifiable audit trail).

The work is directly relevant to Saudi organisations adopting AI under Vision 2030 [9]. The audit and approval features support NCA ECC requirements for event logging, monitoring and access control [11]; taint tracking and identifier masking support PDPL obligations [12]; and human oversight aligns with SDAIA's AI ethics principles of accountability and human-centricity [10] and with the "Govern" and "Manage" functions of the NIST AI Risk Management Framework [18]. Because Mirsad is model-agnostic and deployable as a single service, it suits banks, government service portals, telecom and e-commerce customer support, sectors in which agents would handle Arabic-language customer data.

1.7 Project Timeline (Gantt Chart)

The project follows the two-semester plan in the approved proposal: Senior Project I (Fall 2026) covers investigation, requirements, design and a working prototype; Senior Project II (Spring 2027) covers the full implementation, testing, evaluation and final delivery. Table 1.3 lists the phases with their durations and responsible members, Table 1.4 lists the milestones and the objectives they evidence, and Figure 1.1 shows the schedule, task owners and milestones. Members are abbreviated as BS (Bader Alshuraie), AO (Abdullah Naif Abdullah Al-Orabi), BY (Bader Abdullah Abbod Alyamini), MA (Mohammed Yasser Alanssari) and AB (Abdulmalik bin Saedan); ALL means the whole team. The weekly breakdown against the IS499 deliverables D1–D6 is given in Section 3.4.

Table 1.3: Project phases, durations and responsible members (from the D2 work breakdown)

Phase	Planned dates	Duration	Responsible
Project management (whole project)	1 Sep 2026 – 20 May 2027	37 weeks	AB
1 Investigation and requirements	1 Sep – 25 Oct 2026	8 weeks	ALL (threat model BY, AB; benchmark and dataset design AB, AO)
2 Design	18 Oct – 15 Nov 2026	4 weeks	BS, BY, MA
3 Prototype (Senior Project I)	15 Nov – 24 Dec 2026	6 weeks	BS, BY, AO, ALL
4 Implementation I	10 Jan – 20 Feb 2027	6 weeks	AO, BY, BS
5 Implementation II	1 Feb – 20 Mar 2027	7 weeks	MA, AB, BS, BY
6 Testing and evaluation	10 Mar – 20 Apr 2027	6 weeks	ALL (benchmark AB, detector evaluation AO, penetration test AB, BY, MA)
7 Finalization	10 Apr – 20 May 2027	6 weeks	MA, BS, ALL

The dates are the baseline of the project plan. Actual progress ran ahead of it: on the evidence date of this report (2026-10-14T07:58:56+00:00), before the M1 baseline date, a working and tested prototype of the components planned for phases 3 to 5 already existed (Chapter 4), while the results that depend on team-authored attack data, which M4 and M6 require, were still open. D2 (Section 3.5) records progress against the baseline.

Table 1.4: Project milestones

ID	Milestone	Target date	Objectives
M1	Proposal approved; threat model and SRS agreed	20 Oct 2026	1, 2, 3 (scoping)
M2	Architecture and policy language approved	15 Nov 2026	1
M3	Prototype intercepts MCP traffic with basic policies	20 Dec 2026	1

ID	Milestone	Target date	Objectives
	(Senior Project I demo)		
M4	Taint tracking and Arabic/English detector integrated	20 Feb 2027	1, 2
M5	Approval app, dashboard and audit log complete	20 Mar 2027	1, 3
M6	Benchmark results and security testing complete	20 Apr 2027	2, 3
M7	Final report, poster, presentation and open-source release	20 May 2027	all

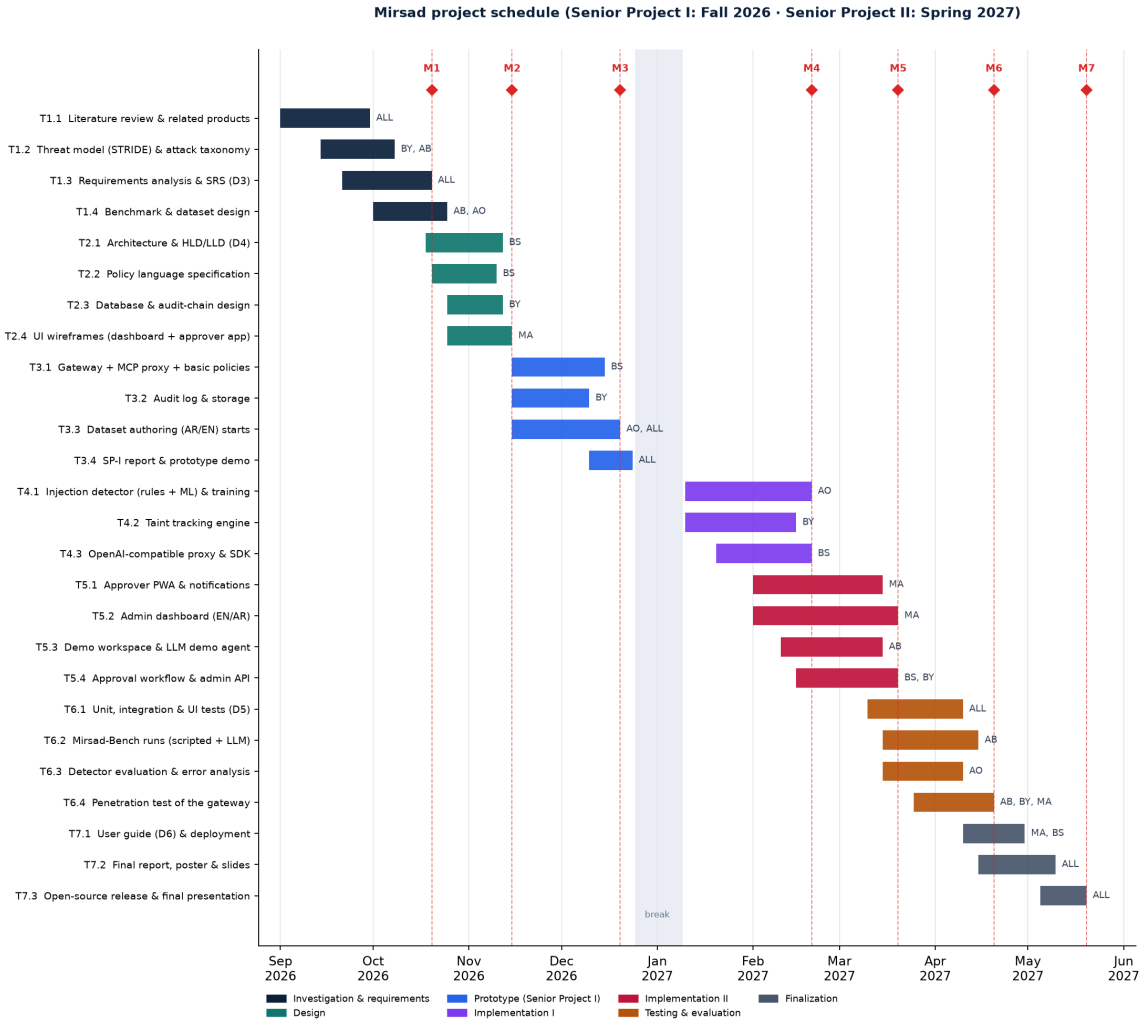


Figure 1.1: Project schedule with task owners (abbreviations as in the text) and milestones M1–M7

Chapter 2 – Literature Review

This chapter reviews the work that Mirsad builds on. Section 2.1 surveys previous studies by theme, Section 2.2 compares existing products and research systems with Mirsad, and Section 2.3 states the gap that Mirsad addresses and the limits of its contribution.

The sources come from six research tracks compiled by the team and fact-checked against primary records (publisher pages, arXiv, standards bodies, official repositories and vendor documentation) on 11 October 2026. Numbers quoted from papers are the values their authors report; the team has not reproduced them, and they are tied to model versions that age quickly. Products are described only as their documentation describes them. Preprints and practitioner blogs are cited where they are the original or most current record of an influential result.

2.1 Overview of Previous Studies

2.1.1 LLM agents and tool use

An LLM agent couples a language model with tools that it can invoke. ReAct [1] established the pattern that most agent frameworks follow: the model alternates between free-text reasoning and actions, and the observation returned by each action is fed into the next reasoning step. On two decision-making benchmarks, ReAct with one or two in-context examples beat imitation- and reinforcement-learning baselines by 34 and 10 absolute points of success rate. Toolformer [2] showed that a model can learn, with little human annotation, when to call an external API, which arguments to pass and how to use the result. Both papers treat tool use as a capability problem, and neither has a threat model for tool outputs: ReAct assumes that observations are honest, and Toolformer's tools are read-only. Nothing in the reason-act-observe loop separates data returned by a tool from instructions, which is the root cause of indirect prompt injection.

Two interfaces carry this loop in practice. OpenAI-style function calling lets the model emit a tool name and JSON arguments that the host application executes. The Model Context Protocol (MCP) [3] standardises discovery and invocation across independent servers: a host uses clients to connect over JSON-RPC 2.0 to servers that expose tools, resources and prompts. The current revision (2026-07-28) is stateless, so each request carries its own protocol version and capabilities. The specification is direct about security. Clients must treat tool annotations as untrusted unless they come from a trusted server, there should always be a human able to deny a tool call, and clients should show tool inputs to the user, validate tool results before passing them to the model and log tool usage for audit. It also says that the protocol cannot enforce these principles and leaves them to implementers. Mirsad can be read as an implementation of these client-side recommendations, placed outside the host so that it works for any agent.

Tool use also does not work equally well in every language. In what its authors describe as the first benchmark of Arabic tool calling, accuracy fell by roughly 5–10% on average when the user wrote in Arabic, whether the tool descriptions were in Arabic or English [19]. Lower accuracy means more malformed or unexpected tool arguments, which per-parameter checks catch whatever their cause.

2.1.2 Prompt injection: origins and taxonomy

Willison named the attack class in September 2022 [20]. He compared it with SQL injection: applications build prompts by concatenating trusted instructions with untrusted input, and the model cannot tell where one ends and the other begins. He suggested that a real fix might resemble parameterised queries but doubted that it was feasible with current architectures. Perez and Ribeiro [21] gave the first systematic study, using their PromptInject framework against GPT-3, and defined two goals that later work adopted: *goal hijacking*, in which the model abandons its task for one chosen by the attacker, and *prompt leaking*, in which it reveals its hidden instructions.

The decisive step for agents was *indirect* prompt injection. Greshake et al. [4] showed that an attacker who never interacts with the model can plant instructions in content that an LLM-integrated application retrieves later, and that the retrieved text can then steer which external APIs the application calls. They built a taxonomy of consequences that includes data theft and self-propagating injections, and demonstrated attacks on Bing's GPT-4-based chat and on code-completion tools. Liu et al. [22] formalised an attack as turning the data of a *target task* into compromised data that carries an *injected task*, showed that earlier techniques are special cases of this framework, and combined them into a stronger attack. Mirsad-Bench mirrors this framing: each attack episode pairs a benign user task with an injection task. OWASP ranks prompt injection first (LLM01) among risks to LLM applications [5], and NIST's taxonomy of adversarial machine learning treats indirect injection as an attack made possible by control over resources that the model reads [23].

Across these sources, attacks can be classified on three largely independent axes, summarised in Figure 2.1. The *channel* is how the malicious text reaches the model: typed by the user (direct injection), planted in data that the agent reads (indirect injection), or hidden in tool metadata. The *technique* is how the attacker obtains compliance. The *goal* is what the attacker wants the agent to do. For a tool-using agent, the goals that matter most are data exfiltration, which needs a flow from a private source to a sink that the attacker can reach, and tool misuse, in which legitimate tools are called with harmful arguments. Indirect injection is the channel Mirsad is built for, because it lets an outsider borrow the user's own privileges, data and tools.

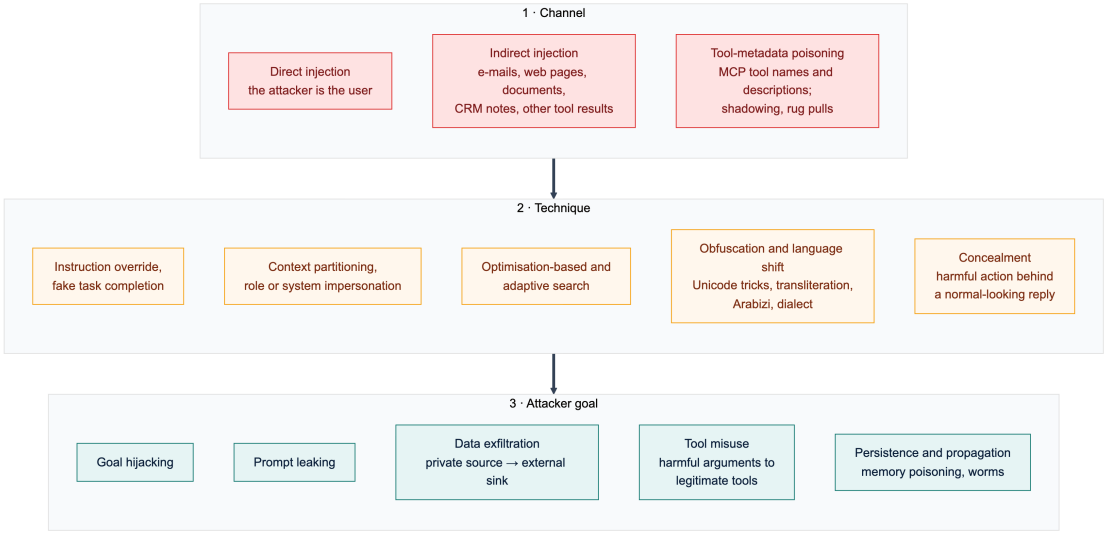


Figure 2.1: A three-axis taxonomy of prompt-injection attacks: the channel that carries the text, the technique used to obtain compliance and the attacker's goal (synthesised from the sources reviewed in Section 2.1.2)

MCP has added channels of its own. Invariant Labs disclosed *tool poisoning* [24], in which an MCP server hides instructions in a tool description that the model reads in full but the user usually does not see. The same disclosure described *shadowing*, where one server's description changes how the agent uses another server's trusted tool, and *rug pulls*, where a description changes after the user has approved it. A later case study [25] described a "toxic agent flow" through the official GitHub MCP server: an issue in a public repository led an agent to copy information from the owner's private repositories into a public pull request. No tool was compromised; the harm came from combining trusted tools with untrusted input. The authors also observed that users often switch tool confirmations to "Always Allow", which argues for asking a human only about risky calls.

Willison's "lethal trifecta" [8] condenses the exfiltration risk: an agent that has access to private data, is exposed to content an attacker controls and can communicate externally can be told to read secrets and send them out, and MCP makes it easy for users to assemble all three without realising it. The risk has already appeared in production. EchoLeak [26] was a zero-click flaw in Microsoft 365 Copilot in which one crafted e-mail caused Copilot to leak internal data. The exploit evaded Microsoft's cross-prompt-injection classifier, bypassed link redaction with reference-style Markdown and used images that the client fetched automatically. Two lessons from EchoLeak shaped Mirsad: a dedicated injection classifier was bypassed in a deployed product, and the fix the authors propose, provenance-based access control, is in effect taint tracking.

2.1.3 Benchmarks for agent security

Benchmarks have moved from text tasks, where success is judged on the model's output, to agents whose success is judged on changes to an environment. Table 2.1 summarises the studies that most influenced Mirsad's evaluation.

BIPIA [27], presented as the first benchmark for indirect injection, found all 25 models tested vulnerable and traced the weakness to models that cannot distinguish informational context from instructions. InjecAgent [7] moved to tool-using agents. It separates *direct harm* through a tool from *data stealing*, a two-step pattern in which private data is first read and then transmitted; the second pattern is exactly the source-to-sink flow that taint tracking is designed to break. AgentDojo [6] replaced fixed test cases with a stateful environment in which success is checked by code against the environment's state, and reports utility, utility under attack and targeted attack success rate (ASR) together. Two of its baseline results are central to Mirsad's design. A *tool filter*, which restricts the agent to the tools a task needs, cut targeted ASR for GPT-4o to about 7% while keeping benign utility (73%, against 69% with no defence). A *prompt-injection detector* reached similar security (about 8%) but cut benign utility from 69% to 41.5%, and utility under attack from 50% to 21.1%, because of false positives. The authors also note that a tool filter fails when the tools a task needs are enough for the attack, as in 17% of their cases. Later benchmarks extended the attack surface to agent memory and planning [28], web agents [29] and MCP tool metadata [30], [31]. In MCPTox the harm is carried out through *legitimate* tools, which is why constraining the parameters of every tool matters more than blocking a suspicious one.

Large red-teaming exercises show that static benchmarks overstate robustness. NIST's Center for AI Standards and Innovation extended AgentDojo with remote-code-execution, database-exfiltration and phishing tasks. Red-teaming with the UK AI Security Institute raised ASR from 11% for the strongest baseline attack to 81%, and running each attack 25 times instead of once raised average success from 57% to 80% [32]. A public competition on *concealed* attacks, in which a harmful action is taken while the final reply looks normal, produced 8,648 successful attacks from about 272,000 attempts against 13 frontier models; every model was vulnerable [33]. When an attack is concealed, the user cannot see it in the chat transcript, so an independent record of tool calls becomes necessary.

Table 2.1: Key studies on agent-security evaluation and their implications for Mirsad

Study	Setting	Main reported finding	Implication for Mirsad
Liu et al. [22]	5 attacks, 10 defences, 10 LLMs, text tasks	No defence sufficient; detectors miss attacks or raise many false alarms	A detector cannot be the only gate
BIPIA [27]	Indirect injection in 5 scenarios, 25 LLMs	All models vulnerable	Enforce outside the model

Study	Setting	Main reported finding	Implication for Mirsad
InjecAgent [7]	1,054 cases; 17 user and 62 attacker tools	ReAct GPT-4 agent compromised in 24% of cases (47% with a fixed prefix)	Data stealing is a source-to-sink flow that can be tracked
AgentDojo [6]	4 stateful suites, 97 user tasks, 629 security cases	Tool filter: about 7% ASR, utility kept. Detector: about 8% ASR, utility 69% to 41.5%	Least privilege first; tune the detector for few false positives
ASB [28]	10 scenarios, 13 model backbones	Highest average ASR 84.30%; memory and planning also attacked	Treat memory writes as sinks
WASP [29]	Web agents in GitLab and Reddit sandboxes	Up to 86% start following injections; at most about 17% complete them	Stop the first unsafe tool call
MCPTox [30]	45 live MCP servers, 353 tools, 20 agents	Highest ASR 72.8%; harm done through legitimate tools	Parameter rules on legitimate tools
MSB [31]	12 MCP attack types, 2,000 instances	MCP-specific attacks missing from earlier benchmarks	Evaluate at the MCP boundary
NIST CAISI [32]	AgentDojo with new high-risk tasks	Red-teaming 11% to 81%; 25 attempts 57% to 80%	Report per task and over repeated attempts
Dziemian et al. [33]	Public competition, 13 frontier models	Every model vulnerable to concealed attacks	Keep an independent audit trail of tool calls

Mirsad-Bench follows the norms these studies set: success is checked on side effects in the environment, utility is reported beside security, false positives on benign tasks are measured, and results are broken down by task and language. It reuses AgentDojo's combination of user tasks and injection tasks on a simulated Saudi company. As far as the team could determine, every benchmark reviewed here is in English; no Arabic benchmark for agent security was found.

2.1.4 Defences at the model, prompt, runtime and architecture levels

Published defences work at four levels, which differ in what an organisation deploying an agent can actually adopt.

Model level. These defences train the model to give less weight to instructions that arrive in data. The instruction hierarchy [34] defines a priority order (system above user above tool output) and trains the model to ignore conflicting lower-priority instructions. SecAlign [35] treats the problem as preference optimisation and reports ASR below 10% even against optimisation-based attacks, at a cost of two to three points on a general-knowledge benchmark. Such defences raise the baseline for everyone who uses the model, but they need the provider's cooperation or open weights and GPU fine-tuning, and they give no guarantee. An organisation that connects a hosted model to MCP servers cannot apply them.

Prompt level. Spotlighting [36] marks untrusted text before it enters the context, by delimiting it, interleaving a marker character through it or encoding it, and tells the model to treat the marked text as data. With GPT-3.5 and GPT-4 it reduced ASR from over 50% to under 2% with little loss of task performance, and Microsoft offers the encoding variant as a preview option of Prompt Shields [37]. Its authors did not test adaptive attackers, and its effect on Arabic script, where marker characters interact with word shape and diacritics, has not been studied.

Runtime detectors. Classifiers screen user input, tool outputs or tool calls. Meta's Llama Prompt Guard 2 [38] is an 86M-parameter classifier with a 512-token window whose model card lists eight supported languages, none of them Arabic, and names vulnerability to adaptive attacks as a limitation. Protect AI's DeBERTa classifier [39] handles English only and is no longer maintained. PIGuard [40] addressed *over-defence*: on 339 benign sentences full of words common in injections, leading guard models fell to roughly 60% accuracy. Among commercial services, Lakera Guard (now Check Point AI Guardrails) screens tool responses and tool descriptions and states coverage of more than 100 languages, Arabic among them [41], while Microsoft states that Azure Prompt Shields was tested with English only [42]. The only open Arabic model found, Ara-Prompt-Guard [43], was trained on 33,000 machine-translated examples, supports Modern Standard Arabic (MSA) only and has no paper or independent evaluation.

The evaluation literature is consistent about detectors. Hackett et al. [44] evaded six deployed detectors, among them Azure Prompt Shield and Protect AI's models, with character-level tricks such as zero-width characters, homoglyphs, bidirectional text and diacritics; one technique reached 100% evasion against every guardrail on each threat type it was tested on. Nasr et al. [45] scaled up gradient, reinforcement-learning, search-based and human red-teaming attacks against twelve recent defences and bypassed most with over 90% success, including Spotlighting (above 95%) and Meta-SecAlign, a successor of SecAlign (96%). PIGuard was the most resistant detector at 71%. Stacking several detectors did not stop adaptive attacks, and many successful triggers did not look malicious, so they could not be caught without also flagging benign inputs.

Runtime frameworks and policy enforcement. NeMo Guardrails [46] wraps an LLM application in programmable rails written in Colang. Its authors say it should not be used as a standalone safety solution, and its documentation warns that tool results are trusted by default [47]. LlamaFirewall [48] combines Prompt Guard 2, an experimental auditor of the agent's reasoning and a scanner for generated code; on AgentDojo it reduced ASR from 17.6% to 1.75%, while utility fell from 47.7% to 42.7%. Two runtime systems enforce rules instead of classifying text. AgentSpec [49] provides a rule language of trigger, predicate and enforcement, with a pause for human inspection as one enforcement action, and prevented unsafe executions in over 90% of code-agent cases with millisecond-level overhead. Progent [50] applies least privilege through default-

deny rules over tool names and arguments, can run as a proxy in front of LLM and MCP endpoints, and sends requests to widen privileges to an approver; it reports AgentDojo ASR falling from 39.9% to 1.0%. Neither system tracks data flow, so neither can tell whether an allowed outbound call carries content supplied by an attacker.

Architecture level. These designs restructure the agent so that untrusted data cannot change its control flow. CaMeL [16] has a privileged LLM write a restricted program from the trusted request alone, a quarantined LLM parse untrusted data without tool access, and an interpreter attach capabilities (provenance and allowed readers) to every value and check policies before each tool call. It completes 77% of AgentDojo tasks with provable security, against 84% for an undefended agent. FIDES [51] labels data for integrity and confidentiality, propagates the labels conservatively through the planner and enforces two generic policies: consequential tools run only from a trusted context, and egress tools send data only to recipients allowed to read it. When a policy is violated, FIDES aborts. Beurer-Kellner et al. [17] generalised these ideas into six design patterns, such as plan-then-execute, while warning that human confirmation is often burdensome and prone to fatigue. A 2026 position paper [52] argues that system-level defences should form the skeleton of agentic systems, with learned models confined to constrained roles and human interaction reserved for ambiguous cases.

For Mirsad, the review leads to the following conclusions. First, detectors are useful signals but weak boundaries: every classifier tested against adaptive attackers was bypassed, and over-defence becomes alert fatigue when a flag triggers a human approval. Second, the strongest results since 2024 come from designs that ask whether an action is allowed given where its inputs came from, rather than whether a text looks malicious. Third, those designs are costly to adopt: CaMeL and FIDES require the agent to be rebuilt. Among the research designs reviewed, only Progent's proxy mode brings deterministic tool-call enforcement to unmodified agents; Section 2.2 shows that some gateway products now do the same for rules on tools and arguments.

2.1.5 Multilingual and Arabic LLM safety

Safety behaviour learned mainly from English data does not transfer evenly to other languages, scripts and dialects. Yong et al. [13] machine-translated harmful instructions into twelve languages and found that low-resource languages, combined, bypassed GPT-4's safeguards about 79% of the time, whereas MSA had a bypass rate of only 3.65%. Deng et al. [14] built MultiJail, unsafe prompts translated by native speakers into nine languages with Arabic among them, and showed that deliberately combining a harmful instruction with a multilingual prompt raised unsafe output to 80.92% for ChatGPT and 40.71% for GPT-4. The most directly relevant result comes from Al Ghanim et al. [15]. GPT-4 and Claude 3 Sonnet mostly refused harmful prompts written in standard Arabic script but produced unsafe output when the same prompts were written in Latin transliteration or Arabizi, the chat style that uses digits such as 3 and 7 for Arabic letters.

The authors attribute this to refusal behaviour tied to particular written forms of words. MSA alone is therefore not the hard case; the less standard forms that Saudi users actually type are.

Arabic safety benchmarks matured quickly in 2025 and 2026, and they point the same way. SalamahBench [53] wrote 8,270 harmful prompts in MSA and five regional varieties, including Saudi Arabic, and found robustness across varieties to depend strongly on the model. ArabicDialectSafety [54] showed that a fine-tuned MARBERTv2 encoder (macro-F1 0.95 on binary safety classification) outperformed prompted frontier LLMs, although its dialects do not include Gulf Arabic. ASAS [55] found obfuscation-based attacks the most effective against Arabic-capable models, and poor agreement between an LLM judge and human annotators. All of these benchmarks measure harmful *requests* to a model; none contains instructions hidden in tool outputs that try to hijack an agent.

Work on Arabic prompt *injection* is sparse. In the only indirect-injection study found that includes Arabic, instructions hidden in about 500 academic papers changed LLM review outcomes when written in English, Japanese or Chinese but had little or no effect in Arabic [56]. The authors do not explain the result, which may reflect weaker instruction-following in Arabic rather than stronger defences; as Arabic-capable models improve, Arabic injections should be expected to become more effective. MIPIAD [57] is the closest published design to a multilingual detector of indirect injection, combining a small Qwen2.5 classifier with lexical features (best F1 0.9205), but it was evaluated only on synthetic English and Bangla data.

For the detector itself, the CAMeLBERT study [58] found that how closely an Arabic encoder's pre-training variety matches the task data matters more than the amount of pre-training data, which is a concern for Gulf dialect and Arabizi. Mirsad's first detector therefore uses character n-grams with a linear model, combined with weighted Arabic and English rules, including rules for Gulf dialect and Arabizi. Before scoring, a normalisation step removes diacritics, elongation and invisible or bidirectional control characters, and folds letter variants, digits and look-alike characters, in response to the evasion techniques above. The choice favours CPU latency and inspectable features. The model is implemented but has not yet been trained on the team's labelled data, so the packaged detector currently runs its rules only; fine-tuning an Arabic encoder is planned as a second stage (Chapter 5).

2.1.6 Standards and Saudi regulation

International frameworks now agree on the controls that agents need. Under LLM01, OWASP's Top 10 for LLM Applications [5] recommends constraining model behaviour, validating outputs with deterministic code, enforcing least privilege, requiring human approval for high-risk actions and segregating untrusted content, and notes that it is

unclear whether fool-proof prevention exists. OWASP's Top 10 for Agentic Applications for 2026 [59] adds risks specific to agents, including agent goal hijack (ASI01) and tool misuse and exploitation (ASI02). NIST's AI Risk Management Framework [18] organises AI risk work into the Govern, Map, Measure and Manage functions, and NIST AI 100-2e2025 [23] recommends designing systems on the assumption that injection will happen, for example by giving access to untrusted data only through well-defined interfaces. With the MCP specification, these documents agree on least privilege, complete mediation, separation of untrusted content, human approval and logging. None of them enforces anything.

Saudi regulation adds concrete obligations. The NCA's Essential Cybersecurity Controls (ECC-2:2024) [11] apply to government entities and operators of critical national infrastructure. Control 2-2-3 requires least privilege, need-to-know and segregation of duties; 2-12-3 requires event logging, continuous monitoring and log retention of at least 12 months; and 4-1 covers third-party cybersecurity, under which third-party MCP servers fall. The Personal Data Protection Law (PDPL) [12], in force since September 2023, requires technical measures to protect personal data (Article 19), requires breaches to be notified (Article 20) and restricts transfers outside the Kingdom (Article 29); its Implementing Regulation [60] sets a 72-hour window for notifying SDAIA of a breach that could cause harm. SDAIA's AI Ethics Principles [10] include privacy and security, reliability and safety, fairness, and accountability, and its Generative AI Guidelines for Government [61] require human review of generative-AI output and keep data classified as restricted or higher out of these tools. These obligations sit within the digital-government goals of Vision 2030 [9]. None of the Saudi documents mentions AI agents or tool calls, so an organisation that deploys agents has to translate them into technical controls itself. Appendix A maps each relevant provision to the Mirsad feature that supports it.

2.1.7 Foundations used by Mirsad

Information-flow control and taint tracking. Denning's lattice model [62] binds every object to a security class and allows a flow only if it respects the ordering of the classes. Labels are combined with a join, so data tends to become more restricted over time, a problem known as label creep. Sabelfeld and Myers [63] surveyed language-based enforcement of noninterference and discussed declassification, the controlled release of information to a lower class. Newsome and Song [64] introduced dynamic taint analysis of unmodified software, with the source, propagation and sink vocabulary that later systems use. All three assume a program whose flows can be analysed. An LLM is opaque and may mix any input into any output, so Mirsad cannot claim noninterference and applies taint at the boundary between tool results and tool calls instead. A session becomes tainted once it has read untrusted content, which is sound but coarse, as in FIDES. Recognisable values such as e-mail addresses, IBANs and record identifiers are

tracked back to the result they came from, which is precise but incomplete. Human approval acts as an explicit, logged declassification step.

Reference monitors. Saltzer and Schroeder's design principles [65] justify the gateway architecture: complete mediation (every tool call passes through the gateway), fail-safe defaults (unlisted tools are denied), least privilege (rules per tool and parameter) and separation of privilege (a human approves high-risk calls).

Tamper-evident logging and policy languages. Schneier and Kelsey [66] combined hash chains with message authentication codes for forensic audit logs and stated the limits of any such scheme: entries written after a compromise cannot be protected, and cryptography can detect deletion but not prevent it. Mirsad's audit log chains events with HMAC-SHA-256 [67]. Because HMAC is symmetric, whoever holds the key could rewrite the chain, and truncation of the newest entries can only be detected against a copy of the head hash kept outside the gateway. For policy, XACML 3.0 [68] separates the policy enforcement point from the policy decision point and the attribute sources that inform it, resolves conflicts with combining algorithms, and lets a decision carry obligations. In Mirsad the gateway is the enforcement point and the YAML policy engine the decision point; the taint tracker and detector supply attributes, and "require approval" behaves like an obligation.

Threat modelling and human-AI interaction. Shostack's method of data-flow diagrams with trust boundaries and STRIDE threat categories [69] structures Mirsad's threat model (Section 3.3.8). For the approval workflow, Amershi et al. [70] give 18 validated guidelines for human-AI interaction, among them making clear why the system acted and supporting efficient correction and dismissal. Parasuraman et al. [71] show that automation can be set to different levels for analysis, decision and action, and warn that supervising automation brings complacency and workload problems. Mirsad therefore automates analysis and decision for most calls and asks a human only about calls that its policy marks as high-risk.

2.2 Comparative Analysis

Table 2.2 compares Mirsad with the commercial products, open-source projects and research systems closest to it. The criteria follow from Section 2.1: where the control sits; whether it enforces action-level rules on tools and their parameters; whether it tracks where data came from; whether its injection detection covers Arabic; whether it offers human approval; whether its log is tamper-evident; and whether it is open source. Each cell records only what the cited sources state as of 11 October 2026. "N/D" (not documented) means that nothing was found in the pages reviewed, not that the capability is absent, and vendor detection claims were not tested by the team. The Mirsad row describes the implemented prototype.

Table 2.2: Feature comparison of runtime guardrails, MCP gateways, research systems and Mirsad (N/D = not documented in the sources reviewed)

System	Deployment point	Action-level policy (tool / parameter)	Taint / data flow	Arabic injection detection	Human approval	Tamper-evident audit	Open source
Lakera Guard / Check Point AI Guardrails [41], [72]	Screening API for model and tool messages	Tool allow-list; parameter rules N/D	N/D	Yes (vendor claim)	N/D	N/D	No
NVIDIA NeMo Guardrails [46], [47]	Library or service around the LLM	Custom rails written in code	No; tool results trusted by default	N/D	N/D	N/D	Yes
Meta LlamaFirewall with Prompt Guard 2 [48], [38]	Library in the agent loop	N/D	No (audits reasoning)	Not listed (8 languages)	N/D	N/D	Yes
Azure Prompt Shields with Foundry agents [37], [73], [42]	Model I/O; agent tool responses (preview)	Tool allow-list; parameter rules N/D	No (Spotlighting marks only)	Tested in English only	Approval primitive; interface left to developer	N/D	No
AWS Bedrock Guardrails with AgentCore Policy [74], [75], [76], [77]	AgentCore Gateway (tool layer)	Yes: Cedar rules, default deny	Partial: session history, no provenance labels	Yes (Standard tier)	Policy pattern; no approval app documented	Partial: opt-in signed log digests	No
Google Model Armor [78]	Model I/O screening API	No per-tool or parameter rules	No (stateless)	Not among tested languages	N/D	N/D	No
Protect AI LLM Guard [79], [39]	Library on prompts and outputs	No	No	No (English only)	No	No	Yes (archived)
Invariant Guardrails and Gateway (Snyk) [80], [81]	LLM and MCP proxy	Yes: tool and argument rules	Partial: call-sequence rules	N/D	N/D	N/D	Yes (maintenance unclear)
Docker MCP Gateway [82]	MCP gateway with	Tool on/off; parameters	No	No detector	N/D	Call logs; integrity	Yes

System	Deployment point	Action-level policy (tool / parameter)	Taint / data flow	Arabic injection detection	Human approval	Tamper-evident audit	Open source
	containerised servers	via custom interceptors				N/D	
agentgateway [83]	MCP and agent proxy	Yes: CEL rules including arguments	No	N/D	N/D	N/D	Yes
IBM ContextForge [84]	MCP federation gateway with plugins	Via OPA plugin on arguments	N/D	English-only plugin	N/D	N/D	Yes
Progent (research) [50]	Library or LLM/MCP proxy	Yes: default-deny argument rules	No	Not applicable	For privilege expansion	N/D	N/D
AgentSpec (research) [49]	Hooks in the agent framework	Yes: trigger-predicate rules	No	Not applicable	Yes (user inspection)	N/D	N/D
CaMeL (research) [16], [85]	Inside the agent (custom interpreter)	Yes: policies on arguments and capabilities	Yes: data-flow graph	Not applicable	Proposed for deployment	No	Yes (unmaintained)
FIDES (research) [51]	Inside the agent (planner)	Trusted-action and permitted-flow policies	Yes: integrity and confidentiality labels	Not applicable	No (aborts)	N/D	N/D
Mirsad	Tool-layer gateway: MCP, OpenAI-compatible proxy, Guard API	Yes: YAML rules, default deny, rate limits	Yes: session and value level (coarser than CaMeL)	Yes: AR/EN rules now, model awaiting training data; F1 pending	Yes: installable bilingual PWA	Yes: HMAC chain per decision, with replay	Yes (Apache-2.0)

The systems in the table fall into five groups, each with characteristic strengths and weaknesses.

Screening services and classifiers (Lakera, Azure Prompt Shields, Google Model Armor, Bedrock Guardrails, Prompt Guard 2, LLM Guard) are easy to adopt, and two of

them (Lakera and the Standard tier of Bedrock Guardrails) are the only products in the sample that document Arabic coverage. However, they return a verdict and leave enforcement to the application, they are bypassed by adaptive attackers (Section 2.1.4), and most were tested only in English. Hosted services also receive every prompt and tool output, which matters under the PDPL's rules on transfers outside the Kingdom.

Guardrail frameworks (NeMo Guardrails, LlamaFirewall) are open source and layered, but security at the tool layer has to be programmed by hand, NeMo trusts tool results by default, components that call an LLM add latency and can themselves be injected, and neither documents an approval workflow or a durable audit store.

Cloud agent platforms (AWS AgentCore Policy, Azure Foundry). AgentCore Policy is the closest commercial analogue to Mirsad: deterministic, default-deny Cedar policies on tool names and input parameters outside the agent's code [74], temporal conditions on session history such as requiring an earlier approval action [75], and CloudTrail log files that can be validated with chained, signed digests [77]. However, it enforces only on traffic routed through an AgentCore Gateway in AWS, documents no provenance labels, models approval as a policy pattern rather than an application for approvers, and offers only opt-in, file-level log integrity. Azure Foundry offers per-tool allow-lists and an approval gate for MCP tools but leaves the approver's interface and the audit trail to the developer [73].

MCP gateways (Docker MCP Gateway, agentgateway, IBM ContextForge) are vendor-neutral, open-source infrastructure with container isolation, tool authorisation and, in agentgateway and ContextForge, rules on tool arguments. None documents taint tracking, an approval channel or a tamper-evident log, and the only documented injection filter wraps the archived English-only LLM Guard model. They are possible hosts for a component like Mirsad rather than complete alternatives to it.

Research policy and information-flow systems (Progent, AgentSpec, Invariant Guardrails, CaMeL, FIDES) give the strongest guarantees. CaMeL and FIDES track data flow precisely, and AgentSpec shows that deterministic rule enforcement, including a pause for human inspection, adds only millisecond-level overhead. On the other hand, CaMeL and FIDES require the agent to be rebuilt, at a cost in utility; Progent, AgentSpec and Invariant's sequence rules do not track where a value came from; research code is often unmaintained; and none of them evaluates the approval interface or documents a tamper-evident record.

Model-level defences such as SecAlign are absent from the table because they are properties of a model rather than components an organisation can deploy. They complement a gateway: a hardened model lowers the rate of attacks that reach the tool boundary, and the gateway contains the ones that do.

2.3 Research Gap and Positioning

The review supports six observations. Detectors and model-level defences are bypassed by adaptive attackers [45], so a classifier cannot be the security boundary. The strongest system-level guarantees require the agent to be rebuilt [16], [51], while deployable tool-layer controls enforce rules on tools and arguments but do not track provenance [50], [74]. Arabic is barely covered, and no Arabic benchmark of indirect injection against tool-using agents was found. Human approval is offered as a primitive rather than a product, and its interface is not evaluated [17]. Audit integrity is an afterthought: only AWS documents cryptographic log validation, at the level of log files. Finally, the MCP specification and Saudi regulations state obligations but leave their enforcement for agents to implementers [3], [11], [12].

Gap statement. As of October 2026, none of the products, open-source projects or research prototypes reviewed documents, in one vendor-neutral and self-hostable component, all five of the following: (1) declarative per-tool and per-parameter policy enforced at the MCP and function-calling boundary; (2) taint labels that follow untrusted tool output to externally visible sinks; (3) an injection detector designed for Arabic as well as English; (4) a ready-to-use human approval channel outside the agent; and (5) a tamper-evident audit trail of every decision. AWS AgentCore Policy covers the first well and the third and fifth partly, but only inside AWS. agentgateway and Invariant cover the first in open source, and Invariant also covers part of the second. CaMeL covers the second most rigorously, but only as a research agent architecture.

Positioning. Mirsad's contribution is integration and localisation rather than a new security primitive. It combines techniques that the literature has validated separately: default-deny rules on tools and parameters in the style of Progent, Invariant and Cedar; taint rules adapted from FIDES's trusted-action and permitted-flow policies; spotlighting as an optional extra layer; Arabic-aware normalisation and Arabizi rules that respond to the documented evasion techniques; human approval treated as a logged declassification step; and an HMAC-chained audit log with decision replay. It places them in one open-source gateway that needs no change to the agent, and evaluates the result on Arabic and English scenarios with a method adapted from AgentDojo. Mirsad does not claim to be the only tool-layer firewall, the only system with per-parameter policy or the only Arabic-capable detector, because AgentCore Policy, agentgateway, Invariant, Lakera and Bedrock Guardrails each contradict one of those claims. Its taint tracking is also weaker than CaMeL's, because a proxy cannot see the data flow inside the model.

Limits. Several limits follow from this position and from the literature reviewed.

- *Coarse taint causes approval friction.* Session-level taint over-approximates, and label creep turns into approval requests; Section 4.2.4 measures this friction on Mirsad-Bench. Whether approvers can handle it without fatigue has not yet been tested with users.

- *Value matching is incomplete.* An agent that paraphrases, translates or re-encodes data defeats value-level matching, and only the session-level rules then apply.
- *The detector is a signal, not a boundary.* Attackers who can query it will find inputs that evade it, as they have for every detector in Section 2.1.4.
- *Detection accuracy and attack success are not yet measured.* Both depend on attack-labelled data written by the team (Section 3.6.4); Sections 4.2.2 and 4.2.3 report them once that data exists.
- *Scope.* Mirsad mediates tool calls only. Backdoors planted during training, supply-chain compromise of MCP servers, tools that bypass the gateway, and harm that occurs only in text, such as a biased summary, are outside its reach.
- *Audit trust.* The HMAC chain reveals tampering only by parties who do not hold the key, and truncation of the newest events needs an external anchor.
- *A moving landscape.* The products compared change month to month, and several of the open-source projects reviewed have been archived or have lost their hosted services. The comparison reflects the documentation available on 11 October 2026.

Chapter 3 – Methodology

This chapter explains how the project was approached, how requirements were obtained, how the system is designed, how it was built and tested, and which ethical, legal and security considerations shaped it.

3.1 Research Design / Project Approach

Mirsad is primarily an **implementation project with a quantitative evaluation**, i.e. a design-science approach [86]: an artefact (the gateway) is built to address an identified organisational problem and is then evaluated against criteria fixed in advance. A purely qualitative design would not answer the central question of Section 1.2, which is about measurable reductions in attack success, false positives and latency. A purely experimental study without a working artefact would not satisfy the IS499 learning outcomes, which require designing, implementing and evaluating an information system.

The evaluation is quantitative and comparative:

- **Controlled experiments on Mirsad-Bench.** The same user tasks and attacks are run against an unprotected agent (baseline) and against the agent behind Mirsad, under several configurations (full protection and ablations with single layers enabled). Two agent types are used: a deterministic *worst-case compliant* agent that always obeys any injected instruction that reaches it, which isolates the effect of the gateway from the model's own robustness, and a real LLM agent (Qwen2.5-7B), so far exercised only in a smoke run of 4 tasks.
- **Classifier evaluation.** The injection detector is evaluated with precision, recall, F1, false-positive rate and ROC-AUC on held-out test sets written separately from the training data, plus, once the team adds it, an external public English dataset for out-of-distribution evidence.
- **Performance measurement.** Added latency is measured over HTTP against a direct tool call.
- **Verification.** Automated unit, integration and API tests, security tests of the gateway itself, and audit-replay checks confirm that the functional and non-functional requirements are met.

The success criteria in Section 1.3 were set in the proposal *before* implementation, which protects the evaluation from being tuned to the results.

3.2 Data Collection / Requirements Gathering

3.2.1 Elicitation methods

Mirsad addresses an emerging threat for which there is no installed user base to observe, so requirements were gathered from several complementary sources rather than from a single existing system:

1. **Document and standards analysis.** The approved proposal (SPAP), the OWASP Top 10 for LLM Applications [5], the MCP specification [3], the NCA Essential Cybersecurity Controls [11], the PDPL [12] and the NIST AI RMF [18] were analysed for obligations that a control point must satisfy (logging, access control, human oversight, personal-data protection).
2. **Literature and benchmark analysis.** Attack taxonomies and threat models from Greshake et al. [4], AgentDojo [6] and InjecAgent [7] were used to derive the attacker goals the gateway must defeat (exfiltration, destructive actions, fraud, phishing, persistence) and the success metrics.
3. **Analysis of existing solutions.** The documentation of commercial and open-source guardrails was reviewed (Section 2.2) to identify missing capabilities, which became the differentiating requirements (action-level policy, taint tracking, Arabic detection, approval, verifiable audit).
4. **Threat modelling.** A STRIDE analysis of the data-flow diagram (Section 3.3.8) produced the security requirements of the gateway itself (authentication, integrity of the audit log, fail-safe approvals, input limits).
5. **Personas and scenarios.** The personas of Section 1.5.1 and end-to-end usage scenarios of the simulated workspace were used to write use cases and to set usability targets such as deciding on an approval in under 30 seconds.
6. **Prototyping and supervisor reviews.** Under the agreed process, requirements are refined iteratively: working increments, starting with the Senior Project I prototype at milestone M3, are demonstrated in the bi-weekly supervisor meetings, and the resulting changes are recorded as backlog items.

Selection criteria. Sources were chosen by three criteria. The first was authority over the target setting: the standards and regulations that a Saudi organisation deploying agents must answer to or is likely to adopt (OWASP Top 10 for LLM Applications, NCA ECC, PDPL, NIST AI RMF and the MCP specification itself). The second was relevance to tool-using agents and *indirect* injection rather than chat-only jailbreaks. The third, for benchmarks, was an evaluation method that judges attack success on the side effects of tool calls and reports utility beside security, as AgentDojo and InjecAgent do, so that the method could be adapted to Arabic scenarios.

Tools. Research notes and the product comparison were kept as Markdown files under version control (`docs/research/`); the use-case, data-flow and threat-model diagrams were drawn in Mermaid (`docs/diagrams/`); and the requirements, use cases and acceptance criteria were written as tables in the report sources, so that the SRS (D3) and this chapter share one copy.

Primary stakeholder data. No interviews have been held and the stakeholder questionnaire in Appendix F has not been administered yet, so the requirements rest on the documentary sources above rather than on practitioner input. The team plans to administer the questionnaire, in particular its Section D, together with the user study described in Section 4.2.8, and to report aggregate results with the number of respondents.

3.2.2 Threat model and assumptions

The gateway protects against an **external attacker who can place content in data sources that the agent reads** (send an e-mail, publish or edit a web page, upload a document, write a review or support ticket, or poison a CRM note through a public form), which is the indirect prompt-injection model of [4]. The attacker cannot modify the agent's code, its system prompt, the gateway or its configuration, and cannot read the gateway's secrets. The attacker succeeds if the agent performs an action that serves the attacker: sending data to an attacker-controlled destination, deleting or corrupting records, transferring money, sending phishing messages, or planting instructions for later. The LLM itself is treated as *untrusted after it has read untrusted content*: Mirsad does not try to make the model robust, it constrains what the model's tool calls can achieve. Insider attacks on the audit log are considered up to the point of an attacker who does not hold the HMAC key.

3.2.3 Use-case model

Figure 3.1 shows the actors and use cases. Besides the four human roles (approver, administrator, auditor and agent developer) the AI agent is modelled as a system actor, and the attacker is modelled as a *misuse actor* following the misuse-case technique: each misuse case is linked to the use case it threatens and to the security use case that mitigates it.

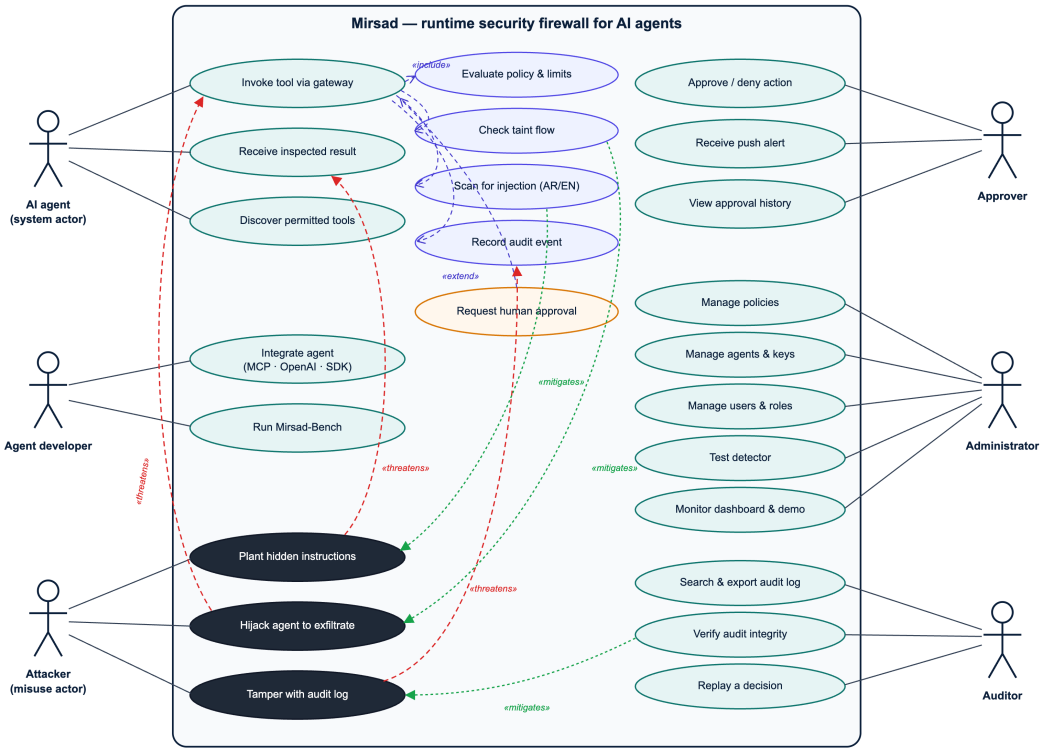


Figure 3.1: Use-case diagram with misuse cases (black) and the security use cases that mitigate them

Table 3.1: Use-case catalogue

ID	Use case	Primary actor	Summary
UC-01	Invoke tool via gateway	AI agent	The agent calls a tool; Mirsad authenticates the agent, evaluates policy, rate limits and taint, possibly asks for approval, executes the call and returns the inspected result.
UC-02	Receive inspected result	AI agent	Mirsad scans the tool result, sanitises or blocks injected instructions, labels trust and sensitivity, and returns spotlighted content.
UC-03	Discover permitted tools	AI agent	The agent lists tools and sees only those allowed for it.
UC-04	Integrate agent	Agent developer	Connect an agent through the MCP proxy, the OpenAI-compatible endpoint or the SDK.
UC-05	Approve / deny action	Approver	Review a pending high-risk action on the phone and decide.
UC-06	Receive push alert	Approver	Get a real-time notification when an action is paused.
UC-07	View approval history	Approver	Review past decisions and notes.
UC-08	Manage policies	Administrator	Edit, validate, activate or roll back the YAML policy.
UC-09	Manage agents and keys	Administrator	Register agents, issue, rotate and revoke API keys.

ID	Use case	Primary actor	Summary
UC-10	Manage users and roles	Administrator	Create approvers, auditors and administrators.
UC-11	Test detector	Administrator	Paste Arabic or English text and see the detector's score and highlighted spans.
UC-12	Monitor dashboard and demo	Administrator	Watch live statistics; run the demonstration agent in protected and unprotected modes.
UC-13	Search and export audit log	Auditor	Filter events and export evidence.
UC-14	Verify audit integrity	Auditor	Recompute the HMAC hash chain and locate any tampering.
UC-15	Replay a decision	Auditor	Re-evaluate a stored decision with its stored policy version and inputs.
UC-16	Run Mirsad-Bench	Agent developer	Reproduce the evaluation with scripted or LLM agents and ablations.
MC-01	Plant hidden instructions	Attacker	Place instructions in an e-mail, web page, document or CRM note the agent will read (mitigated by UC-02).
MC-02	Hijack agent to exfiltrate	Attacker	Make the agent send confidential data or money to the attacker (mitigated by UC-01 policy, taint and approval).
MC-03	Tamper with audit log	Attacker / insider	Modify or delete audit records to hide an incident (mitigated by UC-14).

Table 3.2, Table 3.3, Table 3.4 and Table 3.5 specify the four core use cases.

Table 3.2: Use-case specification UC-01 – Invoke tool via gateway

Field	Description
Actors	AI agent (primary); Approver (secondary, only when escalated)
Preconditions	The agent is registered and holds a valid API key; an active policy exists; the upstream tool server is reachable.
Trigger	The agent's model emits a tool call (MCP `tools/call`, an OpenAI `tool_calls` entry, or a Guard API request).
Main flow	1. Adapter authenticates the agent key and builds a `ToolCall`. 2. Policy engine resolves the tool profile, checks that the tool is allowed, evaluates parameter constraints, rate limits and global rules, and resolves the sink. 3. Taint tracker evaluates the call against the session's provenance state. 4. Gateway combines reasons (deny-overrides) and computes the risk score. 5. The decision is appended to the audit log. 6. The call is forwarded to the upstream tool. 7. UC-02 inspects the result. 8. The inspected result is returned to the agent.
Alternative flows	A1 (deny): at step 4 the action is DENY; the agent receives a bilingual explanation and the call never reaches the tool. A2 (approval): at step 4 approval is required; UC-05 is triggered and the call waits; approved → continue at 6, denied or expired → A1. A3 (monitor mode): the would-be decision is logged but the call is always forwarded.
Exceptions	E1 invalid key → HTTP 401, audited as authentication failure; E2 upstream error → error result returned to the agent and audited; E3 request larger than 1 MB → HTTP 413.
Postconditions	Exactly one `tool_call.decision` event (and, if executed, one `tool_result.inspection` event) exists in the hash chain for the call.

Field	Description
Requirements	FR1.1–FR1.3, FR2.1–FR2.4, FR3.3, FR5.1, FR6.1, NFR1, NFR3

Table 3.3: Use-case specification UC-05 – Approve / deny action

Field	Description
Actors	Approver (primary)
Preconditions	The approver is logged in to the approval app (role approver or admin); a request is pending.
Trigger	The gateway escalates a call (REQUIRE_APPROVAL) and publishes `approval.requested`.
Main flow	1. The app shows a notification and adds the request to the pending list with a risk badge and a countdown. 2. The approver opens it and reads the plain-language summary ("what", "why paused", "where the data came from"). 3. The approver optionally expands the raw arguments. 4. The approver taps Approve or Deny, optionally adds a note, and confirms. 5. The server records the decision, resumes the waiting call and audits `approval.decided`.
Alternative flows	A1: the request expires before step 4 → status EXPIRED, the call is denied. A2: another approver decides first → the app refreshes and shows the outcome.
Postconditions	The approval record stores the decision, the approver's username, the time and the note.
Requirements	FR5.1–FR5.3, NFR4, NFR6, NFR7

Table 3.4: Use-case specification UC-08 – Manage policies

Field	Description
Actors	Administrator
Preconditions	Logged in with role admin.
Main flow	1. Open the Policies page; the active YAML and version history are shown. 2. Edit the YAML. 3. Press Validate; errors are listed with line numbers. 4. Review the diff against the active version. 5. Press Activate and confirm. 6. The new version is stored, the gateway hot-swaps its policy and `policy.activated` is audited.
Alternative flows	A1: validation fails → activation disabled. A2: roll back by activating an earlier version. A3: switch to `mode: monitor` to trial a policy without enforcement.
Requirements	FR2.1, FR2.5, FR7.1

Table 3.5: Use-case specification UC-14 – Verify audit integrity

Field	Description
Actors	Auditor or administrator
Main flow	1. Press "Verify chain" (or run `mirsad audit verify`). 2. The server recomputes the HMAC of every event in sequence order and compares it with the stored hash and the next event's `prev_hash`. 3. The result shows the number of events checked, the head hash, and either "intact" or the first sequence number where the chain breaks.
Postconditions	An `audit.verified` event records the check.

Field	Description
Requirements	FR6.1, FR6.3, NFR3, NFR7

3.2.4 Requirements specification

Functional requirements

Functional requirements are grouped by the eight capabilities in the proposal (FR1–FR8) and refined into testable sub-requirements. Priorities use the MoSCoW scale (M = must, S = should, C = could). Each requirement has an acceptance criterion; the test cases that verify it are listed in the requirements traceability matrix (Appendix B of the final report; deliverable D5, Section B.2).

Table 3.6: Functional requirements

ID	Requirement	Pri.	Acceptance criterion
FR1.1	The gateway shall act as an MCP server towards agents and an MCP client towards upstream MCP servers over stdio and streamable HTTP, forwarding `tools/list` and `tools/call`.	M	An MCP client connected through Mirsad lists and calls upstream tools; results are identical to a direct connection when the policy allows.
FR1.2	The gateway shall expose an OpenAI-compatible `/v1/chat/completions` endpoint that inspects `tool` messages before forwarding and checks returned `tool_calls`.	M	A denied tool call is removed from the response and replaced by an explanation; tool messages are sanitised.
FR1.3	The gateway shall expose a Guard API (`/v1/guard/tool-call`, `/v1/guard/tool-result`, `/v1/tools/{tool}/invoke`) and a Python SDK for frameworks that execute tools themselves.	M	SDK `guard` decorator blocks a denied call by raising `MirsadBlocked`.
FR1.4	Each agent shall only discover the tools its policy allows (least-privilege discovery).	S	`tools/list` and `GET /v1/tools` omit tools with `allow: false`.
FR2.1	The system shall load policies written in a declarative YAML language with per-agent tool entries, glob patterns, defaults, global rules and a monitor mode.	M	All shipped policies validate; invalid documents are rejected with line-numbered messages.
FR2.2	The policy engine shall evaluate per-parameter constraints (`max`, `min`, `max_length`, `enum`, `not_in`, `pattern`, `not_pattern`, `domain_in`, `domain_not_in`, `max_items`, `type`, `required`), each with a configurable violation action (deny or require approval).	M	Each operator has passing and violating test cases.
FR2.3	The policy engine shall enforce sliding-window rate limits per agent and tool.	M	The (n+1)-th call inside the window is denied with code `RATE_LIMIT`.
FR2.4	Reasons from all layers shall be combined with a deny-overrides algorithm, and every decision shall carry machine-readable reason codes and bilingual explanations.	M	A call with one deny reason is denied regardless of other reasons.
FR2.5	Administrators shall be able to validate, activate and roll back policy versions at run time without restarting	S	Activation hot-reloads the policy and creates a `policy.activated`

ID	Requirement	Pri.	Acceptance criterion
	the gateway.		audit event.
FR3.1	Every tool result shall be labelled with a trust level (trusted/untrusted) and a sensitivity level (public/internal/confidential) from its tool profile.	M	The audit event for each inspection records both labels.
FR3.2	The taint tracker shall record identifying values (e-mail addresses, URLs, domains, IBANs, phone numbers, identifiers and text n-grams) from results, with their source tool, call and field.	M	Values embedded in JSON and in free text are both extracted.
FR3.3	The gateway shall escalate or block calls to external sinks after untrusted content was ingested, calls that carry confidential values to external sinks, and calls whose destination value originated in untrusted content.	M	The three taint rules fire on attack fixtures and do not fire when replying to a sender or e-mailing a CRM contact.
FR3.4	After an injection has been detected in a session, high-risk tools shall require approval.	S	`TAINT_AFTER_INJECTION` fires for high/critical tools in that session only.
FR4.1	Every untrusted tool result, and every result of a tool marked `scan` (for example CRM records), shall be scanned for embedded instructions in English, Modern Standard Arabic, Gulf dialect, Arabizi and mixed text, producing a score, label, language and highlighted spans.	M	Detector F1 $\geq$ 0.85 on the held-out Arabic and English test sets.
FR4.2	Text shall be normalised before detection (Unicode NFKC, invisible and bidirectional control characters, Arabic diacritics and letter variants, homoglyphs), and obfuscation shall itself be reported as a signal.	M	Zero-width, homoglyph and diacritic-obfuscated payloads are detected.
FR4.3	Depending on policy, detected content shall be sanitised (offending spans removed), blocked (content withheld) or flagged; untrusted content shall be spotlighted (data-marked) before reaching the model.	M	Sanitised output no longer contains the payload; spotlighting wrapper present.
FR5.1	High-risk actions (payments, deletions, bulk exports, outbound messages under taint) shall be suspended and an approval request with full context created.	M	The call blocks until a decision or timeout.
FR5.2	Approvers shall receive a real-time notification and a plain-language Arabic/English summary that explains what the agent wants to do, why it was paused and where the data came from.	M	Summary mentions the tool, destination and source of tainted data.
FR5.3	Approvers shall approve or deny from a mobile application with an optional note; requests not decided before the timeout shall expire and be denied (fail-safe).	M	Expired requests resolve to DENY; double decisions are rejected.
FR6.1	Every intercepted event shall be appended to an HMAC-SHA-256 hash-chained audit log with secrets and personal identifiers masked.	M	Verification detects modification, deletion and reordering of events.
FR6.2	Users shall search, filter, view and export (CSV/JSON) audit events in the dashboard.	M	Filters by agent, tool, action, type, session, time and text return correct pages.
FR6.3	Any stored decision shall be replayable: the system re-evaluates it with the stored policy version and inputs	M	100% of benchmark decisions replay to the same action.

ID	Requirement	Pri.	Acceptance criterion
	and reports whether the outcome matches.		
FR7.1	Administrators shall manage agents and their API keys (create, rotate, disable), users and roles (admin, approver, auditor), policies and tool registrations.	M	API keys are shown only once and stored hashed.
FR7.2	The dashboard shall show live statistics (calls, blocked, approvals, detections, latency) and a detector playground.	S	Overview reflects new events without reload (SSE).
FR8.1	The system shall ship a simulated enterprise workspace (e-mail, CRM, web, documents, payments) exposed as an MCP server and a demonstration LLM agent.	M	Demo scenarios run in protected and unprotected modes.
FR8.2	The system shall ship Mirsad-Bench, a reproducible benchmark harness with English and Arabic scenarios, ablation configurations and metric reports.	M	One command (`scripts/run_bench.sh`) reproduces the benchmark result tables of the evaluation (final report, Chapter 4; deliverable D5, Section B.5).

Non-functional requirements

Table 3.7: Non-functional requirements

ID	Quality attribute	Requirement	Measure / verification
NFR1	Performance	Median added latency ≤ 200 ms per tool call, excluding time waiting for a human.	Latency benchmark over 1,000 HTTP calls (p50, p95, p99).
NFR2	Interoperability / portability	Model-agnostic: no dependency on a specific LLM provider; standard protocols only (MCP, OpenAI-style JSON).	Same gateway used by an MCP client and by an OpenAI-compatible agent backed by a local model.
NFR3	Security	Agents authenticate with API keys (stored as SHA-256 hashes) and humans with JWTs and role-based access; secrets are never logged; audit integrity is verifiable; security headers, body-size limits and login throttling are applied.	Security test suite (Section 4.2.6 of the final report), redaction tests, SAST and dependency scans.
NFR4	Usability	An approver can understand and decide on a request in under 30 seconds on a phone; UI available in Arabic (RTL) and English.	Keystroke-Level Model estimate and task walkthrough.
NFR5	Deployability	Deployable with a single `docker compose up` (or one script without Docker).	Deployment guide (Appendix E); local run script verified.
NFR6	Reliability / fail-safe	If the approver does not respond, the call is denied; internal errors in a tool never crash the gateway and are reported to the agent safely.	Expiry and executor-failure tests.
NFR7	Accountability	Every allowed, denied or approved action is attributable to an agent, session, policy version and (for approvals) a named human.	Audit event schema; replay test.

ID	Quality attribute	Requirement	Measure / verification
NFR8	Privacy	Personal identifiers (national ID, IBAN, card numbers) are masked in logs; demo data is synthetic.	Redaction unit tests.
NFR9	Maintainability	Modular layers with typed interfaces, ≥ 80% statement coverage of the core package, linted code, documented APIs.	Coverage report, Ruff, OpenAPI docs.
NFR10	Accessibility	Dashboard and approval app follow WCAG 2.2 AA basics: keyboard navigation, labels, contrast ≥ 4.5:1, no colour-only signals, reduced motion.	UI review checklist.
NFR11	Scalability (bounded)	Single node handles ≥ 50 concurrent agent sessions; state is in a relational database (SQLite in development, PostgreSQL in deployment).	Concurrency test in the latency benchmark.

Business rules and constraints

- BR1: Unknown agents and tools not mentioned in the policy are denied (default-deny).
- BR2: Payments and record deletions always require human approval, whatever the risk score.
- BR3: No more than 50 customer records may be exported or e-mailed in one call.
- BR4: E-mail to domains outside the organisation requires approval; data originating from untrusted content may never choose its own destination.
- BR5: An approval request expires after 120 seconds (configurable) and expiry means denial.
- BR6: Audit events are append-only; no user role can edit or delete them through the application.

3.3 System Design & Architecture

This section defines the design of Mirsad as built: its architecture and context, components, core processes, policy language, taint tracking, injection detector, database, security design, interfaces, user interface and deployment. The design follows the engineering contract `docs/dev/BUILD_SPEC.md` (Section 3.4.1), and every diagram was drawn from, or checked against, the code in `src/mirsad/`; the Mermaid sources are kept in `docs/diagrams/`.

Four protection principles [65] guided the design:

- **Complete mediation.** Every tool call and every tool result passes through one enforcement point.

- **Fail-safe defaults.** Unknown agents and undeclared tools are denied, and an internal error, an unavailable audit log or an unanswered approval request ends in denial.
- **Least privilege.** Permissions are granted per agent, per tool and per argument, and an agent only discovers the tools its policy allows.
- **Separation of privilege.** High-risk actions need a human approver, and the auditor role can review the audit log without being able to change policies.

3.3.1 Architectural style and system context

Mirsad is a **policy enforcement point (PEP)** in the sense of the XACML reference architecture [68]. The gateway (`mirsad.gateway.Gateway`) is the PEP and the `PolicyEngine` is the policy decision point; the taint tracker and the injection detector act as attribute providers that supply the provenance of the session's data and evidence about tool results; human approval is an obligation that must be discharged before an escalated call proceeds; and the audit log records every decision. Unlike a conventional PEP, Mirsad mediates in two directions. The **call path** checks what the agent wants to do before the tool runs, and the **result path** inspects what the tool returned before the model reads it, because indirect prompt injection arrives inside tool results [4].

Placing the enforcement point at the tool-calling boundary, rather than inside the model or the agent's code, was the central architectural decision. It makes Mirsad independent of the model (NFR2), requires no change to the agent apart from its tool endpoint, and governs actions rather than text. Designs that restructure the agent, such as CaMeL [16] and the patterns of Beurer-Kellner et al. [17], give stronger guarantees but cannot be added to an existing agent. Privilege-control systems such as Progent [50] are closest to Mirsad's policy layer; Mirsad adds data-flow tracking, Arabic-aware detection, human approval and a verifiable audit trail around such a layer.

Figure 3.2 shows the system context. The agent calls enterprise tools through Mirsad on behalf of a business user; the attacker can only place content in systems the agent reads, such as e-mail, web pages and documents; approvers decide on their phones; and administrators and auditors work in the dashboard.

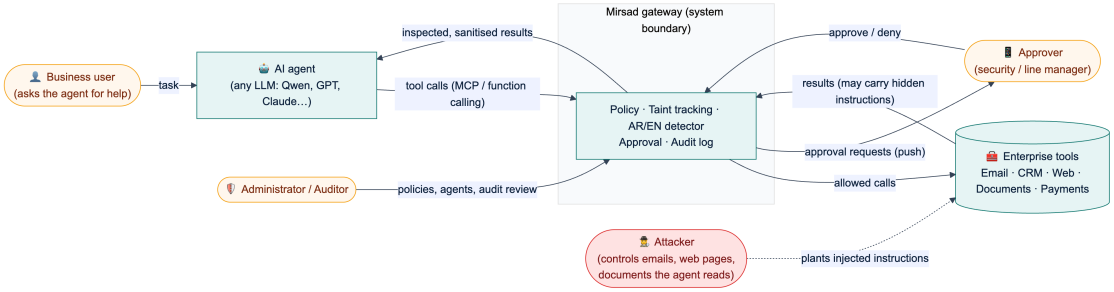


Figure 3.2: System context: Mirsad sits between the AI agent and the enterprise tools; the attacker influences only the content stored in those tools

Figure 3.3 shows the internal architecture, which combines four styles:

1. **Layers.** Protocol adapters (MCP proxy, OpenAI-compatible proxy, Guard API) translate external protocols into one internal model (`ToolCall`, `ToolResult`); the gateway core orchestrates the defence layers; storage sits underneath. A new agent protocol only touches the adapter layer.
2. **Pipes and filters.** On the call path, the policy engine, rate limiter and taint tracker each examine the call and emit `Reason` objects, which a combining step reduces to one decision. On the result path, detection, sanitising or spotlighting, and taint registration run in sequence.
3. **Publish–subscribe.** Approval events and new audit events are published on an in-process notifier and delivered over Server-Sent Events (SSE) and, optionally, Web Push.
4. **Modular monolith.** All components run in one Python process behind one FastAPI application. For a five-person team and a 200 ms latency budget (NFR1), in-process calls were preferred over separate services; typed interfaces and constructor injection (Section 3.3.2) keep the module boundaries, so a layer can later be moved out of process.

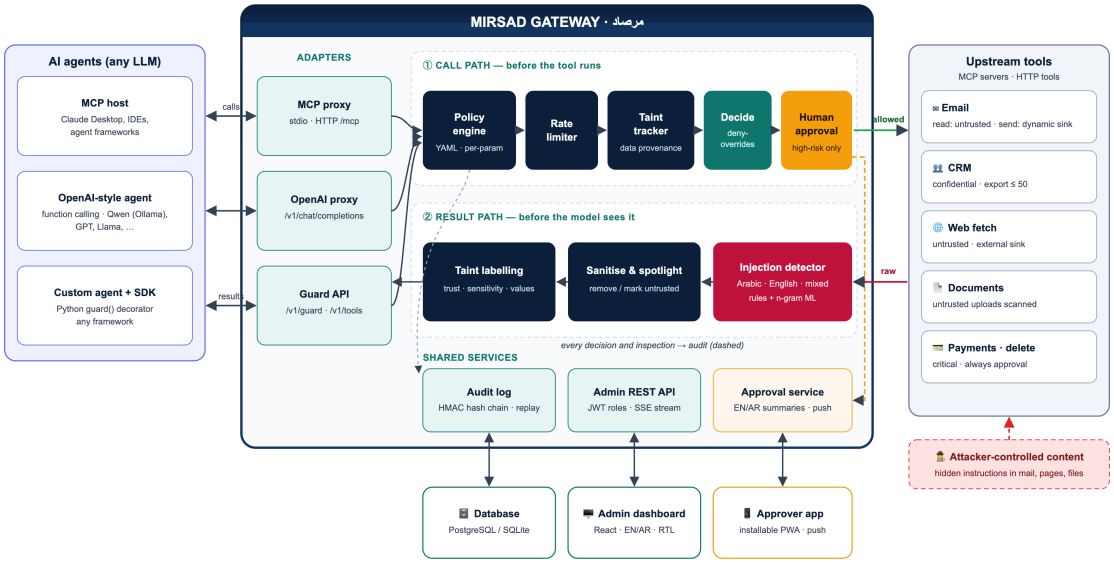


Figure 3.3: Mirsad architecture: protocol adapters, the call path (policy engine, rate limiter, taint tracker, combining step, human approval), the result path (injection detector, sanitising and spotlighting, taint labelling) and the shared services

Table 3.8 records the main design decisions, the alternatives considered and the reasons for each choice.

Table 3.8: Main design decisions

Decision	Alternatives considered	Rationale
Enforce at the tool boundary as a proxy	Guardrail at the model boundary; rebuild the agent around a secure architecture	Works with any LLM and agent; governs actions; inline adapters give complete mediation
Deterministic policy and taint rules as the primary control, the detector as a supporting signal	Rely mainly on an injection classifier	Published detectors are bypassed by adaptive attacks [45]; rules on actions and data flows still hold when text detection misses
Fixed deny-overrides combination with three outcomes	Configurable combining algorithms; first-applicable	Adding a layer or a rule can only make a decision stricter, so independent layers compose safely
Fail closed in every component	Fail open to preserve availability	A control that silently passes calls when it breaks gives false assurance (NFR6)
Template-based bilingual approver summaries	Summaries written by an LLM	A summary built from reason codes cannot be prompt-injected and is reproducible
HMAC hash chain inside the relational database, with optional external anchors	Separate log service; Merkle-tree log; distributed ledger	No extra infrastructure; verifiable offline from an export; a Merkle tree [87] remains an upgrade path
Store each decision with its inputs (policy version, minimal taint snapshot, rate counters)	Store only the outcome	Any decision can be replayed deterministically (FR6.3)
One process with constructor injection; SQLite or PostgreSQL	Microservices; Redis for shared state	Fits the team size and latency budget; injection lets tests and the benchmark swap layers

3.3.2 Components and responsibilities

Figure 3.4 shows the Python packages in `src/mirsad/`, the clients outside the gateway and their dependencies, derived from the import graph of the code; an arrow points from a package to the package it uses. Two dependency rules hold. First, the defence layers (`policy`, `taint`, `detector`) depend only on the shared data model (`mirsad.models`), the text utilities (`mirsad.textutil`) and, for `taint`, the policy's settings types, so each can be tested and benchmarked on its own. Second, the gateway does not construct its collaborators: at start-up `api/components.py` builds the object graph and passes the layers to the `Gateway` constructor, which imports them for type checking only (dashed "injected" arrows). Any component can be replaced through `create_app(gateway_overrides=...)`, which the test suite and the Mirsad-Bench ablations (policy only, taint only, detector only) rely on.

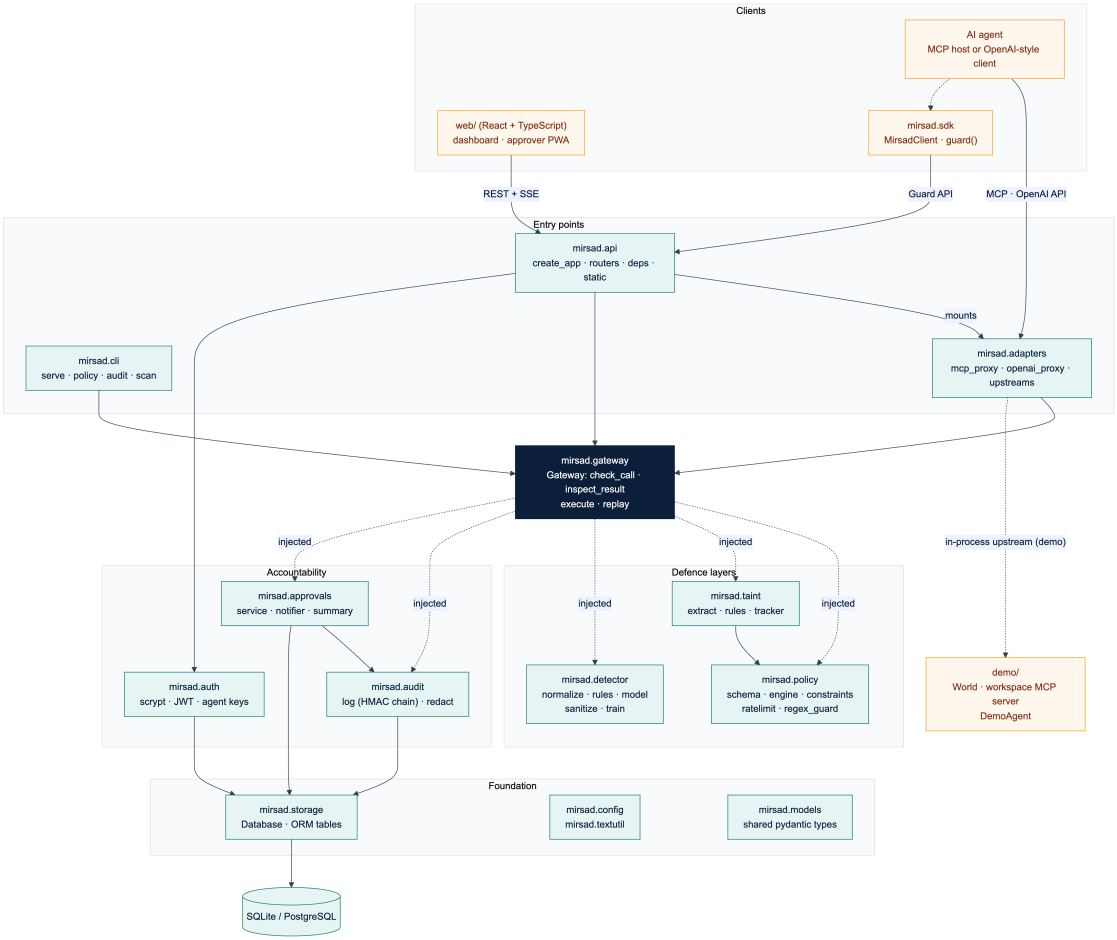


Figure 3.4: Component (package) diagram. Dashed "injected" arrows are runtime collaborators passed to the gateway's constructor; every package also uses the foundation modules

Table 3.9 lists each component's responsibility, main interface and owner (BS = Bader Alshuraie, AO = Abdullah Naif Abdullah Al-Orabi, BY = Bader Abdullah Abbod Alyamini, MA = Mohammed Yasser Alanssari, AB = Abdulmalik bin Saedan).

Table 3.9: Components, responsibilities and interfaces

Component	Responsibility	Main interface	Owner
`adapters`	Translate MCP (stdio, streamable HTTP) and OpenAI chat completions into the internal model; connect to upstream MCP servers and Python tools	`build_mcp_server`, `build_openai_router`, `UpstreamManager`	BS
`gateway`	Decision pipeline: check calls, inspect results, execute allowed calls, replay stored decisions; fail closed	`Gateway.check_call`, `inspect_result`, `execute`, `replay`	BS
`policy`	Parse and validate YAML policies; resolve tool profiles and sinks; evaluate constraints, rate limits and rules	`load_policy`, `validate_policy`, `PolicyEngine`, `RateLimiter`	BS

Component	Responsibility	Main interface	Owner
`taint`	Label results with trust and sensitivity, extract values with their provenance, evaluate the four data-flow rules	`TaintTracker`, `extract_values`	BY
`detector`	Normalise Arabic and English text, score it with rules and a linear model, sanitise and spotlight	`InjectionDetector.scan`, `sanitize`, `spotlight`	AO
`audit`	Redact, chain, store, verify, query and export audit events	`AuditLog`, `verify_events`, `redact`	BY
`approvals`	Approval workflow, bilingual summaries, SSE publish–subscribe and Web Push	`ApprovalService`, `Notifier`, `summarize`	BS, BY (UX: MA)
`auth`	Password hashing, JWTs, agent API keys, user and agent management	`hash_password`, `create_token`, `authenticate_agent_key`	BY
`storage`	Database engine, sessions and ORM tables	`Database`, table classes	BY
`api`	Application factory, routers, authentication dependencies, security middleware, static hosting	`create_app`, `require_user`, `require_agent`	BS
`sdk`	Python client and `guard()` decorator for frameworks that execute tools themselves	`MirsadClient`, `AsyncMirsadClient`	BS
`cli`	Serve, initialise, manage policies, verify, export and replay the audit log, scan text, create agents and users, stdio MCP proxy	`mirsad ...` commands	BS
`web/`	Administration dashboard and approver PWA	REST and SSE client	MA
`demo/`, `bench/`	Simulated Najd Trading Co. workspace and demonstration agent; Mirsad-Bench harness	`World`, `DemoAgent`, `bench.run`	AB

Figure 3.5 shows the main classes and the shared data model. `Gateway` aggregates five runtime services; `PolicyEngine` owns its `RateLimiter`; `ApprovalService` publishes through a `Notifier`; and `UpstreamManager.call` is passed to `Gateway.execute` as the executor of allowed calls, so the gateway never needs to know how a tool is reached. The data classes (stereotype «model») are Pydantic models defined once in `mirsad/models.py`. `ToolCall` and `ToolResult` are the protocol-independent forms of a call and its result. `ToolProfile` is the security profile of one tool for one agent (allowed, trust, sensitivity, sink, risk, approval mode). A `Reason` is one piece of evidence from one layer: a machine-readable code, English and Arabic messages, a proposed `Action`, a risk score and structured evidence. A `Decision` combines the reasons with the risk score, sink, policy version, approval identifier and audit sequence number, and an `InspectionOutcome` is the verdict on a result (pass, flag, sanitize or block) with the content the model will see. `Action` is ordered (allow < require_approval < deny), and `Risk` maps a score to a level (medium from 0.3, high from 0.6, critical from 0.85). Because every layer reports in the same `Reason` type, evidence from different layers can be combined uniformly.

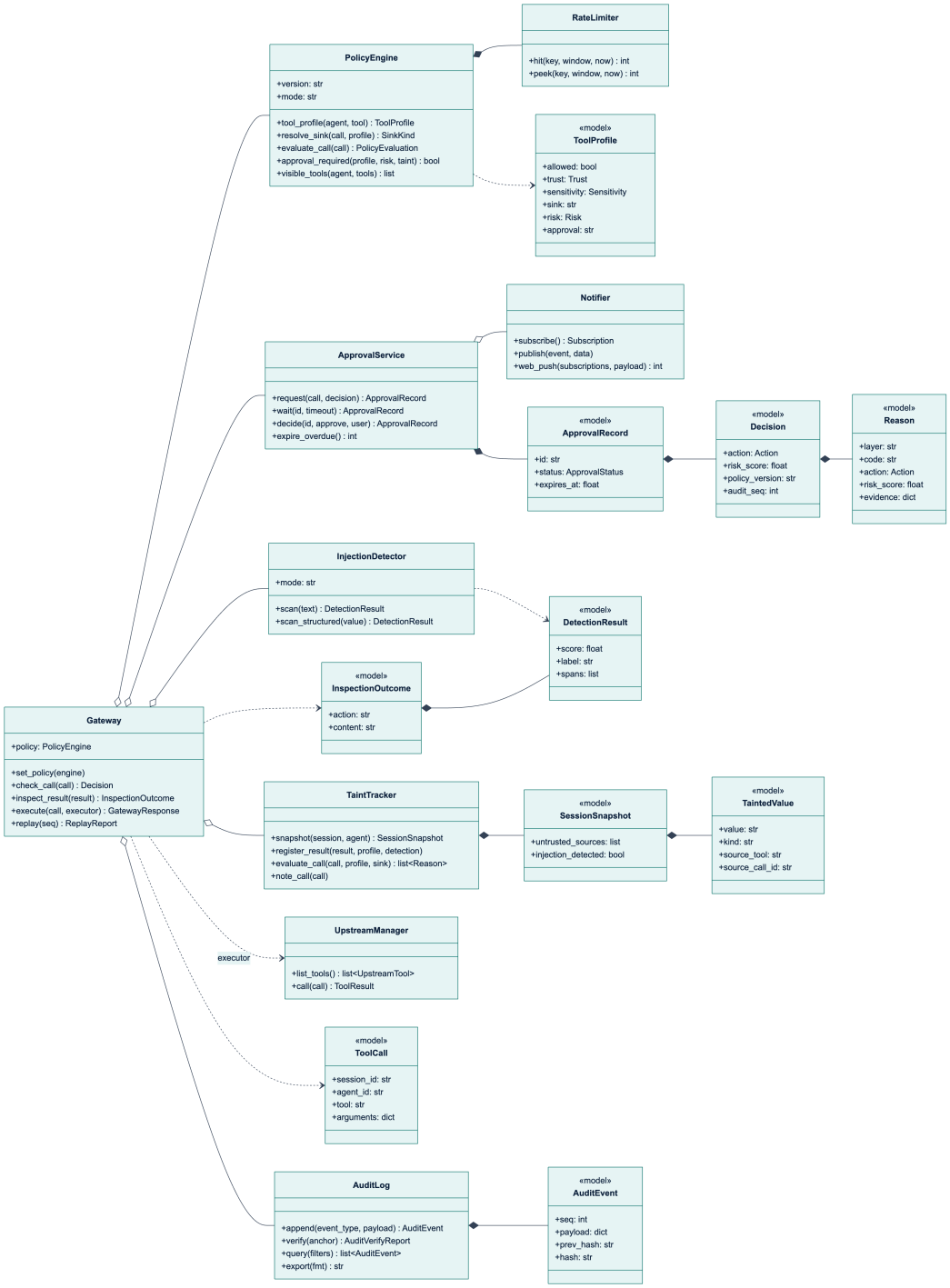


Figure 3.5: Class diagram of the gateway, its collaborating services and the shared data model (signatures simplified; the full interfaces are in the engineering contract)

3.3.3 Core processes

P1: deciding on a tool call. Figure 3.6 shows the sequence for a call arriving through any adapter, and Figure 3.7 shows the decision logic. `Gateway.check_call` performs these steps:

1. Read the session's taint snapshot and call `PolicyEngine.evaluate_call`, which resolves the tool profile and sink and returns the policy and rate-limit reasons (Section 3.3.4).
2. Unless the agent is unknown, call `TaintTracker.evaluate_call` with the resolved sink (Section 3.3.5).
3. Combine all reasons with deny-overrides: the most severe action wins and the risk score is the highest score.
4. If the result is ALLOW but the tool's approval mode requires a human at this risk level, add `APPROVAL_POLICY` and escalate to REQUIRE_APPROVAL.
5. Append a `tool_call.decision` audit event with the call, profile, sink, all reasons, the part of the taint snapshot the rules consulted, the rate-limit counters and the risk score.
6. If approval is required, request it and wait (process P3); the final action is ALLOW only if the request is approved.
7. Record the call in the session and return the `Decision`, whose `latency_ms` excludes time spent waiting for a human.

Steps 1 to 5 run in a worker thread under one re-entrant lock, so snapshot, evaluation and audit append are atomic with respect to concurrent calls while the event loop stays free. Three fail-closed rules apply: an exception in any layer gives DENY with `INTERNAL_ERROR`; a failed audit append gives DENY with `AUDIT_UNAVAILABLE`, because no action may happen without a record; and a failed approval service gives DENY with `APPROVAL_UNAVAILABLE`. In monitor mode the would-be decision is computed and audited with `enforced: false`, but ALLOW is returned and results are not altered, so a policy can be tuned in shadow deployment before it is enforced.

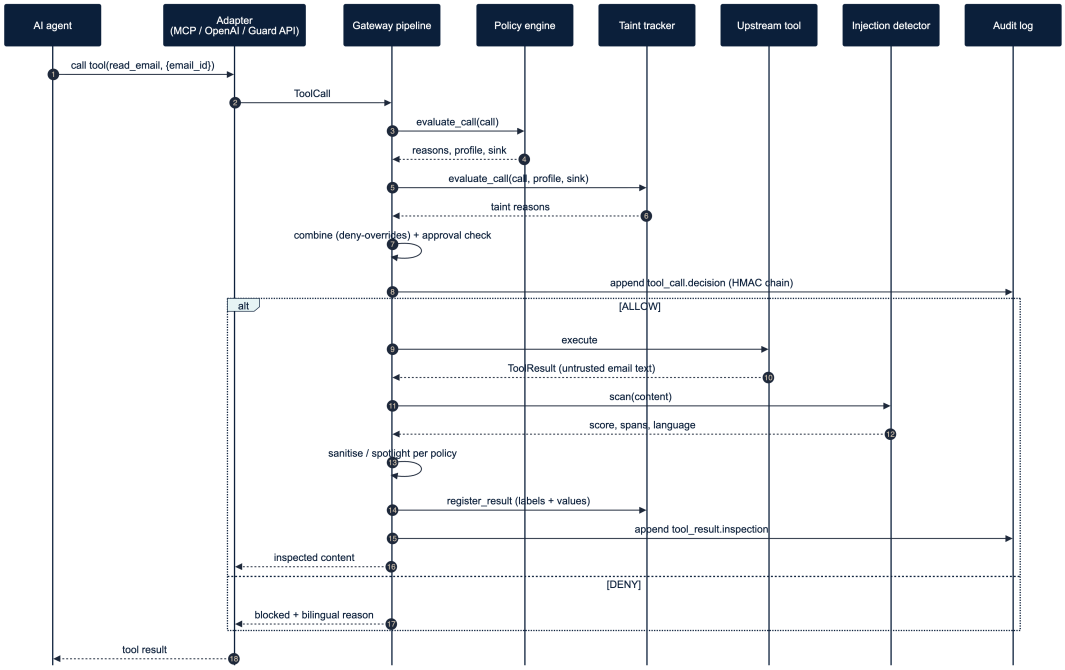


Figure 3.6: Sequence of a tool call through the gateway: decision, execution and inspection of the result

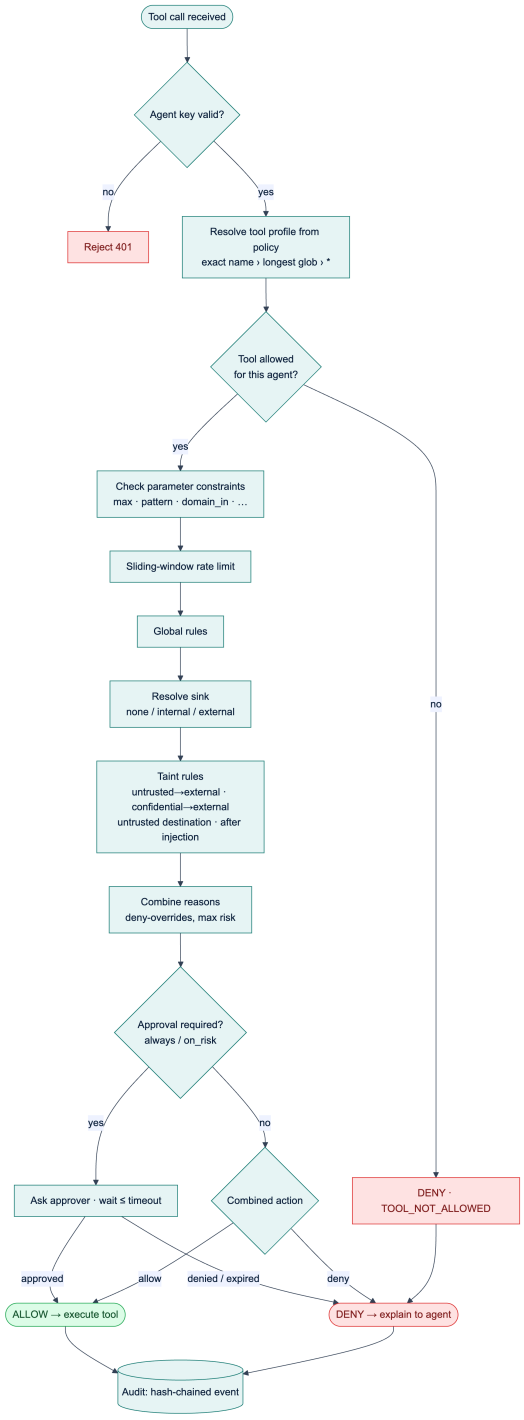


Figure 3.7: Decision logic for a tool call. Constraints, rate limits and global rules all report their reasons before the combining step

P2: inspecting a tool result. `Gateway.inspect_result` scans the content when the tool is marked `scan` or its results are untrusted (results of tools the policy does not allow are always untrusted), maps the score with the tool's thresholds to pass, flag, sanitize or

block, spotlights untrusted content that is not blocked, registers the result with the taint tracker and appends a `tool_result.inspection` event. A sanitiser that cannot locate the suspicious text escalates to block, and a detector failure withholds the result. `Gateway.execute` composes P1 and P2: it runs the tool (with a 120 s default timeout) only for allowed calls and re-binds the result to the original call and session, so that an upstream cannot attribute its output to another call.

P3: human approval. Figure 3.8 and Figure 3.9 show the approval process and its state machine. `ApprovalService.request` stores a pending record with the redacted arguments, the decision, English and Arabic summaries and an expiry time (120 s by default), publishes `approval.requested` over SSE and, when configured, Web Push. The waiting call blocks on an event object, with a database re-check every 5 s for decisions made by another process. Every transition is a conditional `UPDATE ... WHERE status = 'pending'`, so exactly one decision wins when two approvers act at once, and a second decision is rejected with HTTP 409. At the deadline the request becomes EXPIRED, which the gateway treats as DENY; this also prevents a late approval of an action the agent was already told had been refused.

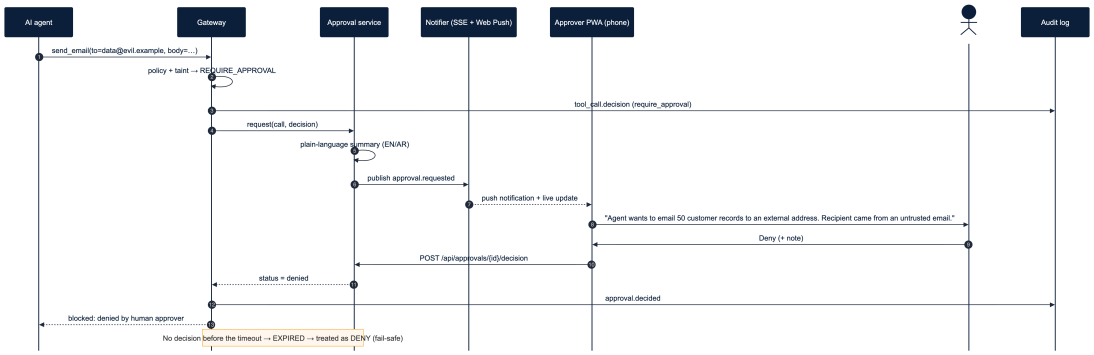


Figure 3.8: Sequence of an escalated call: approval request, notification, human decision and resumption of the waiting call

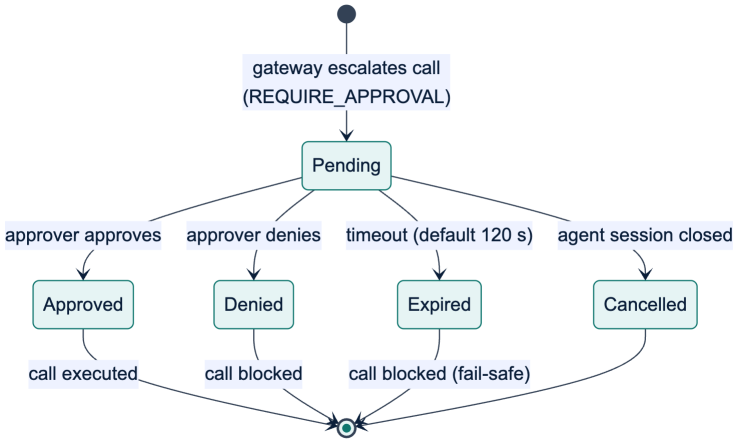


Figure 3.9: State machine of an approval request; every terminal state other than Approved blocks the call

The approver's summary is built by `approvals/summary.py` from the reason codes and their evidence, not by a language model, so attacker-controlled text cannot rewrite it; values copied from arguments are length-limited and stripped of line breaks. The summaries below were produced by this code for a benign case, in which an agent that has read an external supplier's e-mail replies to that supplier.

> The Email & CRM Assistant wants to send an email to accounts@red-sea-logistics.example (outside Najd Trading Co.) using the send_email tool. Mirsad paused it because the agent read an untrusted email (read_email call_r1) earlier in this session, and this action sends data outside the organisation, and the recipient address is outside the allowed domains (najd-trading.example). Risk level: high.

يريد الوكيل «Email & CRM Assistant» إرسال بريد إلكتروني إلى accounts@red-sea-logistics.example (خارج شركة نجد للتجارة) عبر الأداة send_email. أوقف مرصاد الطلب مؤقتًا لأن الوكيل اطّلع في هذه الجلسة على بريد إلكتروني غير موثوق (read_email call_r1)، وهذا الإجراء يرسل بيانات إلى خارج المؤسسة، ولأن عنوان المستلم خارج النطاقات المسموح بها (najd-trading.example). مستوى الخطورة: مرتفع.

The example shows a deliberate trade-off. The recipient came from the e-mail's `from` field, which the policy lists as trusted, so the rule that denies destinations chosen by untrusted text does not fire; the session-level taint rule and the domain constraint still ask a human to confirm. This is safe but adds friction to legitimate work.

P4: audit verification and replay. Verification recomputes the hash chain (Section 3.3.7). `Gateway.replay(seq)` loads the policy version recorded in a decision event, rebuilds the call and the stored taint snapshot, re-runs the policy with the stored rate counters (leaving the live limiter untouched) and the taint rules on the snapshot, and compares the action and reason codes; inspection events are rescanned. Because the taint

rules are a pure function of the call and the snapshot, replay is exact (Section 4.2.7 reports the replay results).

Data-flow view. Figure 3.10 is the context-level data-flow diagram, and Figure 3.11 decomposes the gateway into six processes: 1.0 authenticate and adapt, 2.0 evaluate the policy, 3.0 check taint, 4.0 obtain approval, 5.0 inspect the result and 6.0 audit. Data stores D1 (policies), D2 (audit events) and D3 (approvals) are database tables. D4 (session taint state) is kept in memory per process, bounded in size and expired after six idle hours; the part of D4 a decision depends on is copied into its audit event, so the audit log, not memory, is the durable record.

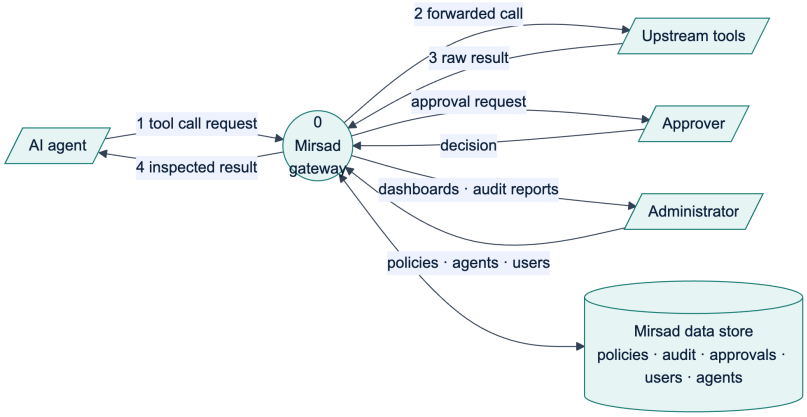


Figure 3.10: Context-level (level 0) data-flow diagram of Mirsad

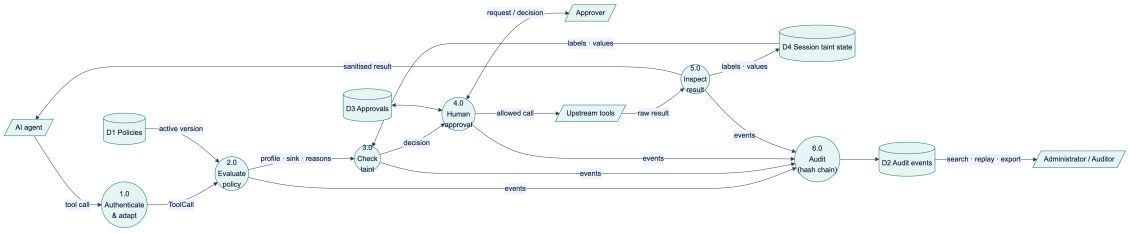


Figure 3.11: Level 1 data-flow diagram: processes 1.0 to 6.0 and data stores D1 to D4

3.3.4 Policy language design

The policy language had four goals: a policy must be reviewable by a security analyst in a pull request; permissions must default to deny and reach individual arguments; mistakes must be caught with precise messages before activation; and evaluation must be deterministic and versioned so that decisions can be replayed. General-purpose languages such as Rego [88] or XACML [68] are more expressive, but a domain-specific YAML format with a strict Pydantic schema fits a prompt-injection policy on one page and is harder to get dangerously wrong. Table 3.10 summarises the structure of a policy document.

Table 3.10: Structure of a policy document (schema version 1)

Element	Keys	Meaning
Document	`version`, `mode` (enforce or monitor), `organization` (`name`, `internal_domains`)	Schema version; shadow mode; domains inside the organisation
`defaults`	`decision` (deny, allow or require_approval), `approval_timeout_s`	Treatment of unmatched tools (deny by default); time a human has to decide
`detector`	`flag_threshold`, `block_threshold`, `on_detect` (sanitize, block or flag), `spotlight`	Global detector settings; agents and tools can override them
`taint`	`enabled`, `destination_params`, `trusted_fields`, one action per taint rule	Data-flow settings (Section 3.3.5); agents can override them
`agents[]`	`id`, `name`, `description`, `tools` (tool key → tool entry)	One entry per agent identity; `id` equals the agent's `policy_agent_id`
Tool entry	`allow`, `trust`, `sensitivity`, `sink` (none, internal, external or dynamic), `sink_params`, `risk`, `approval` (never, on_risk or always), `scan`, `constraints`, `rate_limit`, `detector`	Security profile of one tool for one agent
Constraint	`max`, `min`, `max_length`, `min_length`, `enum`/`in`, `not_in`, `pattern`, `not_pattern`, `domain_in`, `domain_not_in`, `max_items`, `required`, `type`, `on_violation`	Checks on one argument, combined with AND; a violation denies or requires approval
`rules[]`	`id`, `when` (`tool`, `agent`, `arg_matches`, `sink`, `session_tainted`, `min_risk`), `action`	Global rules evaluated after the tool entries

The excerpt below, from the tool entries of `configs/policies/default.yaml`, shows how the business rules of Section 3.2 are expressed: e-mail outside the organisation needs approval (BR4), exports are capped at 50 records (BR3), payments always need approval (BR2), and unlisted tools are denied (BR1).

```
send_email:
  allow: true
  sink: dynamic
  sink_params: [to, cc, bcc]
  risk: medium
  constraints:
    to:      {domain_in: [najd-trading.example], on_violation: require_approval}
    body:    {max_length: 20000}
  rate_limit: {max_calls: 20, per_seconds: 3600}
get_customer:     {allow: true, sensitivity: confidential, scan: true}
export_customers:
  allow: true
  sensitivity: confidential
  scan: true
  risk: high
  approval: on_risk
  constraints:
    count: {max: 50, on_violation: deny}
  rate_limit: {max_calls: 3, per_seconds: 3600}
transfer_funds:   {allow: true, risk: critical, approval: always, sink: external,
                   constraints: {amount_sar: {max: 10000, on_violation: deny}}}
"*":              {allow: false}
```

CRM results are trusted and confidential, but they are scanned (`scan: true` on `search_customers`, `get_customer` and `export_customers`), because their free-text notes can contain text that came from web forms or customer e-mails; a detection relabels the result as untrusted (Section 3.3.5).

Resolution. For a tool name the engine selects the exact key, otherwise the longest matching glob, otherwise the catch-all `"*"`, otherwise `defaults.decision`; equal-length globs that can match the same name are rejected as ambiguous. A `dynamic` sink is EXTERNAL if any recipient or URL in the destination arguments has a domain outside `internal_domains` or no recognisable domain, INTERNAL if all destinations are internal, and NONE if there is none.

Evaluation and combining algorithm. `PolicyEngine.evaluate_call` collects reasons in a fixed order: (1) an unknown agent is denied and evaluation stops; (2) a tool that is not allowed is denied and evaluation stops; (3) an informational `TOOL_RISK` reason carries the tool's risk score (0.1, 0.3, 0.6 or 0.9 for low to critical); then (4) every argument constraint, (5) the sliding-window rate limit, (6) every matching global rule and (7) `approval: always`. After step 3 nothing short-circuits, so the audit log and the approver see every objection. The gateway adds the taint reasons and applies **deny-overrides**: for the reasons r1, …, rn of a call, the final action is the most severe among them (deny > require_approval > allow) and the risk score is the maximum of their scores. The approval mode is applied last: `always` escalates every call; `on_risk` escalates when the combined risk is at least 0.6, or when the tool's own risk is high or critical and a taint rule objected; `never` does not escalate, although a deny still denies. Deny-overrides is monotonic, so adding a layer, constraint or rule can only make a decision stricter. The accepted limitation is that an `allow` rule cannot create an exception to a deny; exceptions are written as narrower deny rules. Table 3.11 lists the reason codes outside the taint layer, whose codes are given in Section 3.3.5.

Table 3.11: Reason codes of the policy, detector, approval and system layers

Code	Layer	Action	Risk score
`UNKNOWN_AGENT`, `TOOL_NOT_ALLOWED`	policy	deny	0.5
`TOOL_RISK`	policy	allow (informational)	0.1 / 0.3 / 0.6 / 0.9
`CONSTRAINT_VIOLATION`	policy	the constraint's `on_violation`	0.5
`RATE_LIMIT`	ratelimit	deny	0.5
`RULE_MATCH`	policy	the rule's `action`	rule's score, else 0.5 (0 for allow)
`APPROVAL_ALWAYS`	policy	require approval	the tool's risk score
`APPROVAL_POLICY`	approval	require approval	combined score
`APPROVED_BY_HUMAN`,	approval	allow, deny, deny	unchanged

Code	Layer	Action	Risk score
DENIED_BY_HUMAN, APPROVAL_EXPIRED			
INTERNAL_ERROR, AUDIT_UNAVAILABLE, APPROVAL_UNAVAILABLE	system, approval	deny (fail closed)	1.0
MONITOR_MODE	system	allow (records the would-be action)	0
INJECTION_SUSPECTED, INJECTION_DETECTED	detector	deny when the result is blocked; otherwise allow, with the result flagged or sanitised	detector score
SCAN_FAILED	detector	deny (result withheld, fail closed)	1.0
RESULT_TRUNCATED	system	allow (informational)	0

Safe validation. The schema is strict: unknown keys, operators and duplicate YAML keys are errors, with "did you mean" suggestions and the YAML line and column. Because policy regular expressions run against arguments written by a possibly hijacked model, the validator rejects patterns with nested ambiguous quantifiers, the classic cause of catastrophic backtracking (ReDoS); at run time at most 50,000 characters reach a policy regex, and a truncated input counts as a rule match, so the check fails closed.

Versioning and hot reload. A policy's version is the SHA-256 hash of the canonical JSON of the parsed document, so formatting does not change it but any semantic change does. Every version is stored, and the version in force is recorded in every audit event. Engines are immutable: activation swaps in a new engine atomically, and an evaluation in progress keeps the engine it started with.

3.3.5 Taint-tracking design

An LLM can mix any input into any output, so taint cannot be propagated through the model, and the guarantees of language-based information-flow control are out of reach [63]. Mirsad therefore tracks data at the boundary it controls, in the style of dynamic taint analysis [64]: tool results are **sources**, tool calls are **sinks**, and the rules look for the combination of private data, untrusted content and external communication that makes an agent exploitable [8]. The approach follows information-flow control for agents [51], [89], except that a violation can be routed to a human approver instead of always aborting.

Labels. Every result receives two labels from its tool profile, as in Denning's lattice model [62]: integrity (trusted or untrusted, where untrusted means a third party can control the content) and confidentiality (public < internal < confidential). Detector evidence overrides declared trust: a result labelled as an injection is registered as

untrusted even if its tool is declared trusted. Each recorded value also carries a `trusted_field` flag for fields vouched for by the producing system rather than the content's author, such as the sender, identifier and date of a received e-mail. This separates "reply to the sender", which is normal, from "send to an address written in the body", the typical exfiltration pattern.

Two granularities. At **session level**, a session that has read untrusted content stays tainted for the rest of its life, a conservative but sound assumption. At **value level**, recognisable values are extracted from every result with their source tool, call and field: e-mail addresses, URLs, domains, IBANs, phone numbers, numbers of at least six digits, record identifiers and, for confidential results only, hashed word 8-grams of long text. Results and arguments are canonicalised the same way (Unicode NFKC, invisible characters removed, Arabic-Indic digits converted, case folded, Saudi mobile numbers unified), so a value matches whatever its spelling. N-grams are hashed with BLAKE2b, which keeps them stable across processes for replay and avoids storing the text. Table 3.12 lists the four rules evaluated before every call.

Table 3.12: Taint rules evaluated for each tool call

Rule (reason code)	Fires when	Default action	Risk
`TAINT_UNTRUSTED_TO_EXTERNAL`	The session has read untrusted content and the call's sink is external	require approval	0.6
`TAINT_CONFIDENTIAL_TO_EXTERNAL`	An argument contains a value recorded from a confidential result and the sink is external (sending customers only their own record is exempt)	require approval	0.7
`TAINT_UNTRUSTED_DESTINATION`	A destination argument (recipient, URL, IBAN, webhook) or its domain first appeared in untrusted content outside a trusted field, and never in a trusted result; the organisation's own domains are excluded	deny	0.9
`TAINT_AFTER_INJECTION`	An injection was detected earlier in the session and the tool's risk is high or critical	require approval	0.8

Every taint reason names the source tool, call and field of the matching value, so the approver sees where the data came from. State is bounded (at most 2,000 values per session; sessions dropped after six idle hours), and the rules are one pure function of the call, profile, sink, settings and a read-only view of the session, used by both live evaluation and replay.

The coarse session-level rule is the price of soundness; Section 4.2.4 measures the approval friction it causes. Value-level matching, in turn, can be defeated by paraphrasing or re-encoding data, which is why the session-level rule and human approval remain as a backstop.

3.3.6 Injection detector design

The detector reads untrusted tool results and the results of tools marked `scan: true`, such as CRM records, whose free-text notes can contain text that came from outside the organisation. It is designed as a **signal, not a security boundary**, because adaptive attacks have bypassed published detectors with high success rates [45]; the policy and taint layers are built to block harmful actions even when the detector misses. The detector catches common cases cheaply, drives the result-path actions, arms `TAINT_AFTER_INJECTION` and gives approvers and auditors readable evidence. It works in six stages:

1. **Normalisation.** HTML entities are decoded; zero-width and bidirectional control characters removed; text converted with NFKC and lower-cased; Arabic diacritics (tashkeel) and elongation (tatweel) removed; alef, ya and ta marbuta variants and Persian look-alikes unified; digits converted to ASCII; Cyrillic, Greek and Armenian look-alikes in Latin words folded; and letter-spaced words joined. Obfuscation is reported as *signals*, and an offset map takes each span back to the original text, so highlights and sanitising point at the characters the user and the model see.
2. **Language identification** (English, Arabic, mixed or other, including Arabizi), for reporting and per-language evaluation only.
3. **Rules.** 158 weighted regular-expression rules in YAML: 76 English, 51 Arabic (Modern Standard Arabic and Gulf, with some Egyptian and Levantine forms), 7 Arabizi and 24 language-independent rules for structural tricks such as chat-template tokens, hidden HTML, markdown-image exfiltration and encoded blobs. The rule score combines the best weight of each distinct rule by noisy-OR, and evasion signals raise it only when a rule has fired, because ordinary Arabic text produces some of them.
4. **Model.** Hashed character 2- to 5-grams (2^20 features) and word 1- to 2-grams (2^18 features) feed a class-balanced logistic regression. Texts are scored over windows of 600 characters with a stride of 300 and the highest window probability is kept, so one injected sentence in a long e-mail is not diluted.
5. **Ensemble.** The score is a noisy-OR of model score m and rule score r, $1 - (1 - m)^{\alpha} (1 - r)^{\beta}$, calibrated so that the operating point chosen on the validation split (highest F1 with a false-positive rate of at most 5%) maps to 0.5; the policy defaults (flag at 0.5, block at 0.8) keep their meaning after retraining.
6. **Persistence without pickle.** The model is stored as plain arrays and JSON metadata, loaded with `allow_pickle=False` and checked for shape and finite values; if it is missing or invalid, the detector runs in rules-only mode instead of failing.

A linear model over hashed n-grams was chosen over a fine-tuned transformer because the detector runs inline on CPU, the team's dataset is small, and character n-grams

tolerate Arabic morphology, spelling variation and Arabizi (Section 3.4.5). On the result path, scores below the flag threshold pass, scores between the thresholds are flagged and audited, and higher scores trigger the tool's `on_detect` action. Sanitising widens each span to its sentence or line and replaces it with a bilingual marker; a positive detection without a span fails closed and the content is withheld. Untrusted content that reaches the model is spotlighted [36]: it is wrapped in delimiters whose boundary is derived from a SHA-256 hash of the call identifier, source and content, so the content cannot contain its own closing marker, and a bilingual note tells the model to treat it as data.

The packaged detector currently contains no trained model and runs in rules-only mode, which is also the mode recorded by the latency benchmark (`rules_only`). The F1 scores on the held-out English and Arabic test sets (target 0.85) are produced by running `make detector` after the team has added its labelled data as described in `docs/detector.md`, and Section 4.2.2 reports them. The attack-labelled data, like the attack files of Mirsad-Bench (`bench/scenarios/attacks/README.md`), is written by the team (Section 3.6.4).

3.3.7 Database design

Persistence uses SQLAlchemy 2.0 with typed declarative mappings and only portable column types, so the same schema runs on SQLite in development and PostgreSQL 16 in deployment. SQLite connections enable foreign keys and write-ahead logging, so dashboard readers never block the audit writer. Timestamps are UNIX epoch seconds stored as floats, matching the Pydantic models. Figure 3.12 shows the six tables.

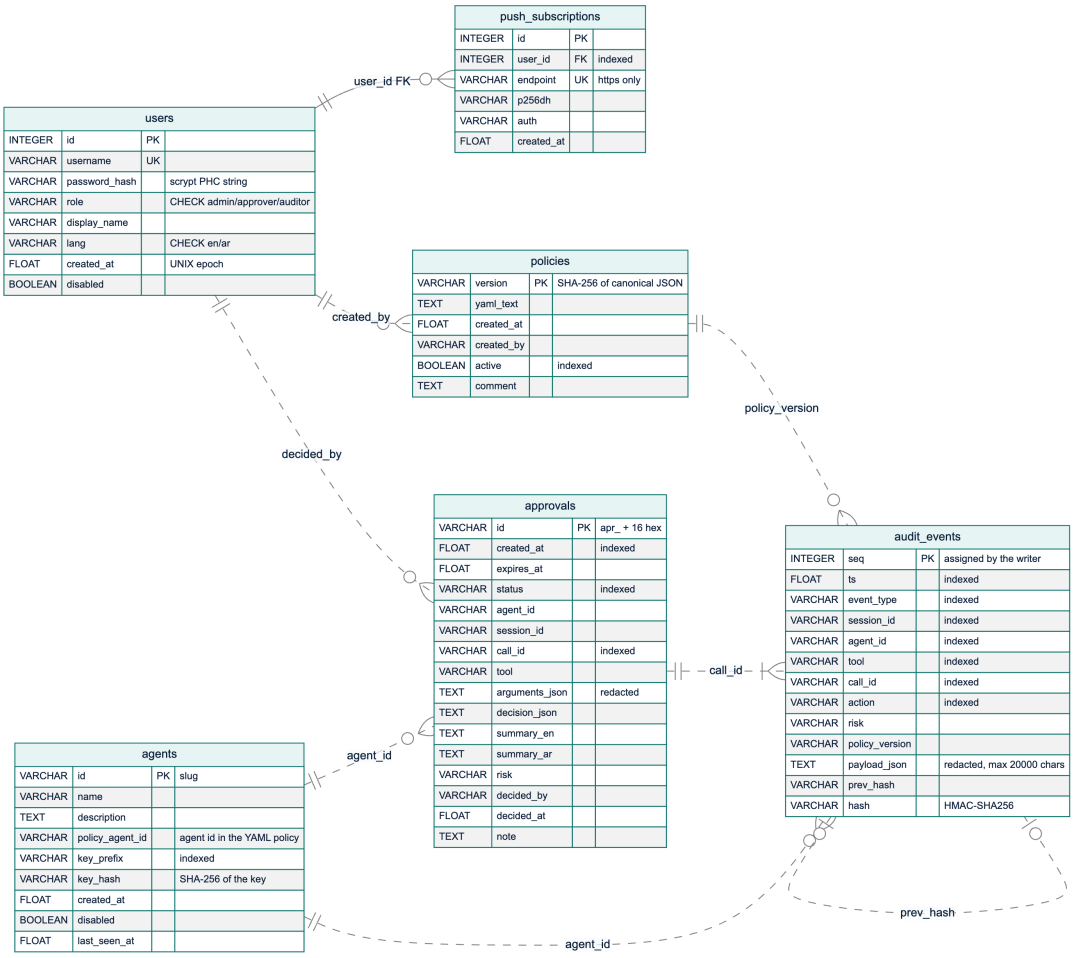


Figure 3.12: Entity-relationship diagram of the six tables. The solid line is the only enforced foreign key; dashed lines are logical references by value

Only `push_subscriptions.user_id` is an enforced foreign key (to `users.id`, with cascading delete). The other relationships are logical references by value: `audit_events.agent_id` and `approvals.agent_id` hold an agent's policy identifier; `approvals.decided_by` and `policies.created_by` hold usernames; `audit_events.policy_version` holds a `policies.version`; an approval and the audit events of its call share the `call_id`; and each event's `prev_hash` equals the previous event's hash. These references are deliberately not enforced: the audit log is append-only evidence that must outlive the deletion or renaming of what it mentions, and its hashes cover values exactly as written, so a cascading delete or a blocked insert would destroy or prevent evidence.

The management tables are in third normal form. Three columns hold JSON documents by design (`audit_events.payload_json`, `approvals.decision_json`, `approvals.arguments_json`): their shape depends on the event type and they must be stored verbatim to be hashed and replayed, while frequently filtered fields are promoted

to indexed columns. An approval copies the agent, tool and arguments of its call, so the record shows exactly what the approver saw. Table 3.13 is the table dictionary.

Table 3.13: Data dictionary of the database tables

Table	Purpose	Key and constraints	Written by / read by
`users`	People who sign in	`id` primary key; `username` unique; role restricted to admin, approver, auditor; language to en, ar	Administrators / authentication, user management
`push_subscriptions`	Browser push endpoints per user	`id` primary key; `endpoint` unique (HTTPS only); `user_id` foreign key with cascade	Approver PWA / Web Push delivery
`agents`	Agent identities and key hashes	`id` (slug) primary key; `key_prefix` indexed; only the SHA-256 hash of the key is stored	Administrators / agent authentication
`policies`	Every policy version stored	`version` (SHA-256) primary key; `active` indexed	Policy activation / gateway start-up, replay, administrators, auditors
`audit_events`	Hash-chained log of decisions, inspections, approvals, administrative changes and logins	`seq` primary key assigned by the writer (no auto-increment); seven indexes	`AuditLog.append` only / administrators, auditors, statistics, replay
`approvals`	Approval requests and outcomes	`id` primary key; indexes on `status`, `created_at`, `call_id`	Approval service / approvers, administrators, waiting calls

Hash-chain design. Each event is a link in a chain built from HMAC-SHA-256 [67], following the hash linking of Haber and Stornetta [90] and the secure audit log of Schneier and Kelsey [66]:

```
hash(n)      = HMAC-SHA256(K_audit, prev_hash(n) + "|" + canonical_json(event(n) without
"hash"))
prev_hash(n) = hash(n-1),   prev_hash(1) = "0" * 64
K_audit      = HMAC-SHA256(secret_key, "mirsad/audit-hmac/v1")
```

Canonical JSON uses sorted keys, no whitespace and UTF-8, so Arabic text is hashed as written. The hashed body contains every stored column, including `seq` and the indexed columns, so editing any field breaks the chain, and the audit key is separate from the JWT signing key although both derive from the server secret. Before hashing, `redact()` removes credentials and masks Saudi national ID and Iqama numbers, IBANs and card numbers to their last four characters (Section 3.6.1), so secrets never enter the chain; payloads are limited to 20,000 characters while keeping their structure. There is one writer per process: under a lock it reads the head and inserts `seq = head + 1` in the same transaction, and if another process takes that number first, the primary key rejects the insert and the writer retries.

Verification walks the table in sequence order and reports the first broken event: a gap in `seq` reveals a deletion, a `prev_hash` that does not link reveals an insertion or reordering, and an HMAC mismatch reveals a modification or a hash forged without the key. The same function verifies an unfiltered JSON export offline. A rewrite by someone holding the key, and truncation of the newest events, cannot be detected from the chain alone; for both, `AuditLog.anchor()` returns the current head to be stored outside the database, and verifying against an earlier anchor detects them. Because enforced calls are refused when the append fails, no action can happen without an event.

3.3.8 Security design and threat model

A firewall concentrates trust, so the gateway itself is a target. The threat model of Section 3.2 was analysed by applying STRIDE to each element of the data-flow diagram [69], [91]. Figure 3.13 shows the five trust boundaries: untrusted internet content (A), the agent runtime, whose model may be hijacked (B), the gateway (C), the enterprise systems (D) and the human approvers (E). Table 3.14 lists the main threats, the mitigation in the design and the test that verifies it. Tests ST-01 to ST-23 are in `tests/security/test_gateway_security.py`; other entries name the test modules of the component concerned.

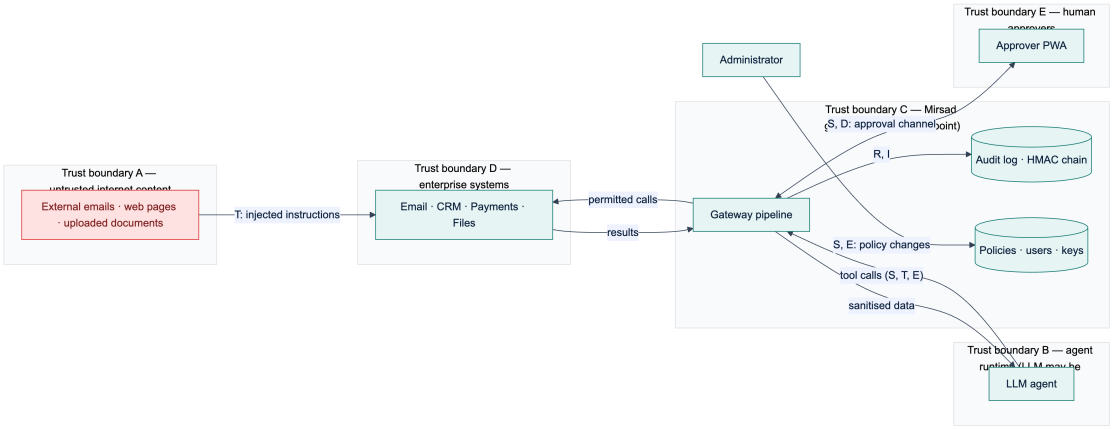


Figure 3.13: Threat model with trust boundaries A to E and the STRIDE categories of each data flow

Table 3.14: STRIDE analysis of the gateway

	Threat	Element	Mitigation	Verified by
S	Tools called with a guessed, stolen or revoked agent key	Agent APIs, `/mcp`	Per-agent keys with 256-bit secrets, stored as SHA-256 hashes, compared in constant time; rotation revokes the old key	ST-04, ST-05
S	Forged, unsigned or expired session	Human APIs	HS256 JWT with pinned algorithm,	ST-01, ST-02

	Threat	Element	Mitigation	Verified by
	token		expiry, issuer and role checks; user re-loaded on every request	
S	Online password guessing	Login	scrypt hashes; 5 failures per username and IP (30 per IP) in 5 minutes return HTTP 429	ST-19
T	Audit records edited or deleted to hide an incident	`audit_events`	HMAC hash chain, verification, external anchors	ST-06, ST-07
T	Injected instructions in a tool result steer the agent	Result path	Detector with sanitise, flag or block; spotlighting; detected injection marks the session	`tests/api/test_gateway.py`, `tests/detector/`
T	A policy that hangs the engine (ReDoS) is activated	Policy engine	Admin-only activation; validator rejects nested ambiguous quantifiers; input bound	ST-08
R	An approver denies having approved an action	Approvals	Decider, time and note stored; first decision final; `approval.decided` audited	ST-09
R	Login attempts leave no trace	Authentication	Failed logins audited with the username, never the password	ST-10
I	A hijacked agent sends customer data or money outside the organisation	Call path	Domain constraints; taint rules on untrusted and confidential flows; approval for payments	`tests/taint/`, `tests/policy/`; attack success rate pending (Chapter 4)
I	A URL tool is used to reach internal services (SSRF)	`fetch_url` policy	URL pattern constraints in the default policy	ST-23
I	Secrets in tool arguments leak into the audit log or exports	Audit log	`redact()` before hashing and storage	ST-11
I	Hashes, keys, internal errors or files leak through the API	API	No hashes in listings; generic 500 responses; static files confined to the web root	ST-12, ST-13, ST-14
I	The dashboard is framed or called from another site	Web front end	Same-origin CSP, frame denial, CORS allow-list, no tokens in URLs	ST-15, ST-16
D	Oversized	API, tools	1 MB body limit;	ST-18, `tests/policy/`

	Threat	Element	Mitigation	Verified by
	requests or call flooding		sliding-window rate limits per agent and tool	
D	No approver answers and the call hangs	Approvals	Expiry after the timeout, treated as denial	`tests/audit/test_approval_service.py`
E	A non-admin role performs administrative actions or alters its role claim	Role checks	Role-based access on every route; signed claim; role re-read from the database	ST-03, ST-20
E	An agent key is used on human APIs, or a user token on agent APIs	Authentication	Separate credential types per route group	ST-21
E	A hijacked or unknown agent calls tools outside its role	Policy engine	Default deny; least-privilege tool discovery	ST-22

Further controls apply across components: every component fails closed (Section 3.3.3); searches use parameterised ORM queries, which ST-17 checks with injection-style input; responses carry security headers (content security policy, `nosniff`, frame denial, referrer policy, HSTS over HTTPS); and the server secret is read from the environment or generated into an owner-only file. The security suite contains 23 security tests (45 test cases); Bandit reported 0 issues over 13,023 lines, and dependency audits found 0 vulnerable Python packages and 0 npm vulnerabilities (Section 4.2.6).

These tests verify the gateway's own defences. Whether the design stops hijacked agents is measured by the reduction in attack success rate on Mirsad-Bench relative to an unprotected agent (target at least 80%; Section 4.2.3), once the team adds its attack files to `bench/scenarios/attacks/` (Section 3.6.4).

3.3.9 Interface design

Mirsad has three kinds of external interface: agent interfaces, a REST and streaming interface for people, and the command line. The agent interfaces differ in who executes the tool, which determines the strength of enforcement (Table 3.15). In **inline** interfaces Mirsad executes the tool itself, so a denied call cannot run. In **advisory** interfaces the agent runtime executes tools and Mirsad removes or refuses denied calls; this protects against a manipulated model but assumes the agent's own code is honest, so inline interfaces are recommended when the runtime is not trusted.

Table 3.15: Agent-facing interfaces

Interface	Transport	Tool executed by	Enforcement
MCP proxy	stdio (`mirsad mcp-proxy`) or streamable HTTP at `/mcp`	Mirsad	Inline
Tool invocation	`POST /v1/tools/{tool}/invoke`, `GET /v1/tools`	Mirsad	Inline
OpenAI-compatible proxy	`POST /v1/chat/completions`	Agent runtime	Advisory: denied calls removed from the model's response
Guard API and Python SDK	`POST /v1/guard/tool-call`, `POST /v1/guard/tool-result`; `MirsadClient.guard()`	Agent runtime	Advisory: the SDK raises `MirsadBlocked` before the tool runs

MCP proxy. One MCP server is built per agent. `tools/list` returns only the tools the agent's policy allows, and every `tools/call` goes through `Gateway.execute`. Any refusal or internal failure returns an MCP error result with a bilingual explanation whose reason texts are clipped, so a block message cannot re-inject a long attacker-controlled value into the model's context. Only inspected text is returned; MCP structured content is never forwarded, because the sanitiser does not rewrite it.

OpenAI-compatible proxy. For each request the proxy (1) replaces every `role: tool` message with its inspected version, (2) removes tool definitions the policy does not allow, (3) forwards the request to the configured LLM endpoint without the agent's key, and (4) checks every returned tool call, removing denied calls, appending a bilingual explanation and setting `finish_reason` to `stop` when no call remains. Streaming is rejected, because a tool call must be complete before it can be checked.

REST and streaming API. People authenticate with a JWT from `POST /api/auth/login`; agents use their API key in the `Authorization` or `X-Mirsad-Key` header. Agent keys are rejected on `/api/*` and user tokens on `/v1/*`. Table 3.16 groups the endpoints. Bodies are JSON, errors have the form `{"detail": ...}`, status codes follow HTTP semantics (401, 403, 404, 409 for a conflicting decision or duplicate, 413 for bodies over 1 MB, 422, 429), and an OpenAPI description is generated from the typed models. Live updates use SSE on `GET /api/stream`, read with `fetch` and an `Authorization` header so that tokens never appear in URLs. The `mirsad` command line covers operations that should not depend on the web interface, such as policy activation, audit verification, export and replay, and agent and user creation.

Table 3.16: REST API overview

Group	Endpoints	Principal	Purpose
Health and authentication	`GET /healthz`; `POST /api/auth/login`; `GET /api/auth/me`	none; any user	Liveness and version; sign-in;

Group	Endpoints	Principal	Purpose
			current user
Agent gateway	`/v1/guard/tool-call`, `/v1/guard/tool-result`, `/v1/tools`, `/v1/tools/{tool}/invoke`, `/v1/chat/completions`, `/mcp`	agent key	Check, inspect, discover and execute tool calls
Approvals	`GET /api/approvals`, `GET /api/approvals/{id}`, `POST /api/approvals/{id}/decision`	admin, approver	Pending queue, history and decisions
Live updates	`GET /api/stream` (SSE); `GET /api/push/vapid`, `POST /api/push/subscribe`, `POST /api/push/unsubscribe`	any user	Approval and audit events; Web Push subscriptions
Audit	`GET /api/audit/events`, `GET /api/audit/events/{seq}`, `POST /api/audit/verify`, `POST /api/audit/events/{seq}/replay`, `GET /api/audit/export`	admin, auditor	Search, verify, replay, export (JSON or CSV)
Policies	`GET /api/policies`, `GET /api/policies/{version}`, `POST /api/policies/validate`, `PUT /api/policies`, `POST /api/policies/{version}/activate`	admin; auditor read-only	Versions, validation, activation, rollback
Agents and users	`/api/agents` (list, create, update, delete, rotate key); `/api/users` (list, create, update, delete)	admin	Identity management; keys shown only once
Tools, statistics, detector	`GET /api/tools`; `GET /api/stats/overview`, `GET /api/stats/timeseries`; `POST /api/detector/scan`	admin, auditor; any user	Tool profiles; dashboard figures; detector playground
Demonstration	`/api/demo/chat`, `/state`, `/reset`, `/scenarios`, `/scenarios/{id}/apply`	any user, demo mode only	Simulated workspace and demo agent

3.3.10 User-interface design

Mirsad has two front ends built from one React and TypeScript code base: the **administration dashboard** at `/` and the installable **approver PWA** at `/approve`. They share the API client, live-stream reader, translation dictionaries, explanation components and design tokens, and the gateway serves both from its own origin, so a strict content security policy that allows only same-origin scripts holds. The interface follows these principles:

1. **Plain language first, evidence on demand.** A request opens with three short sections, *What the agent wants to do*, *Why Mirsad paused it* and *Where the data came from*, followed by the raw arguments, collapsed by default. This applies the human–AI interaction guidelines of explaining why the system acted and supporting

efficient correction [70]. Only escalated calls reach a human, to limit the alert fatigue and complacency that automation research warns about [71].

2. **Bilingual and right-to-left by design.** Every string exists in English and Modern Standard Arabic with the same keys, which tests enforce. Choosing Arabic sets the document's language and direction, and CSS logical properties mirror the whole layout without separate stylesheets. Identifiers such as tool names, addresses, hashes and YAML stay left-to-right inside isolation elements, and server messages are bilingual at the source (`message_ar`, `summary_ar`).
3. **Accessibility to WCAG 2.2 level AA** [92]. Pages use semantic landmarks; every control has a label; everything works from the keyboard; dialogs trap and restore focus; live regions announce new requests; colour is never the only signal; contrast is checked by a test; reduced-motion preferences are respected; and touch targets in the approver app are at least 44 × 44 pixels.
4. **Safe, deliberate actions.** Approve and Deny open a confirmation sheet that restates the consequence and takes an optional note; the buttons are disabled when a request has expired or the phone is offline; API keys and generated passwords are shown once; and text from agents and tools is always rendered as text, never as HTML.
5. **Speed for the approver.** The target is a decision in under 30 seconds on a phone (NFR4): pending requests are sorted by soonest expiry with a countdown, and a push notification opens the request directly.

The dashboard has ten pages (Overview, Activity and audit, Approvals, Policies, Agents, Users, Tools, Detector playground, Demo console, Settings); the sidebar shows only the pages the signed-in role may use, and the API rejects any request outside that role. Figure 3.14 shows the audit-event drawer with its hash-chain fields and replay and verification actions, and Figure 3.15 shows the policy editor in Arabic with the mirrored layout. Figure 3.16 and Figure 3.17 show the approver app's pending list in Arabic and a request in English. The screenshots were taken from the running system with the simulated workspace; Appendix C contains the complete set.

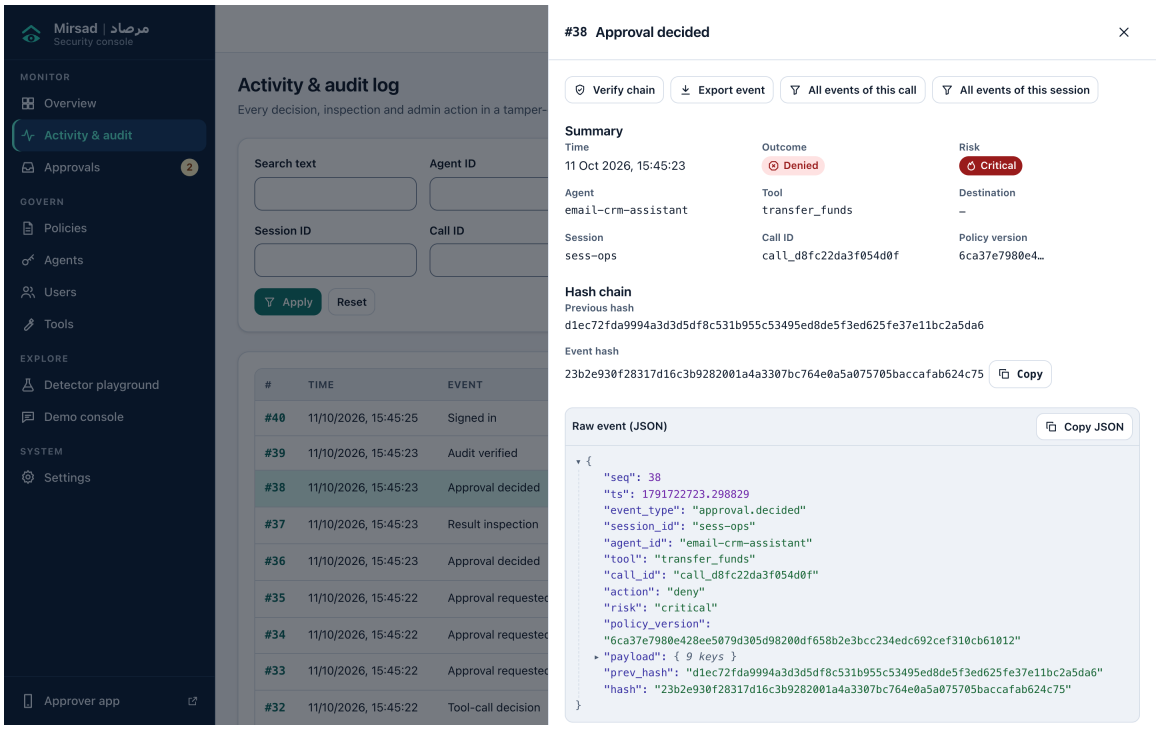


Figure 3.14: Activity and audit page with the detail drawer of an approval decision: summary, previous hash and event hash, and the raw event

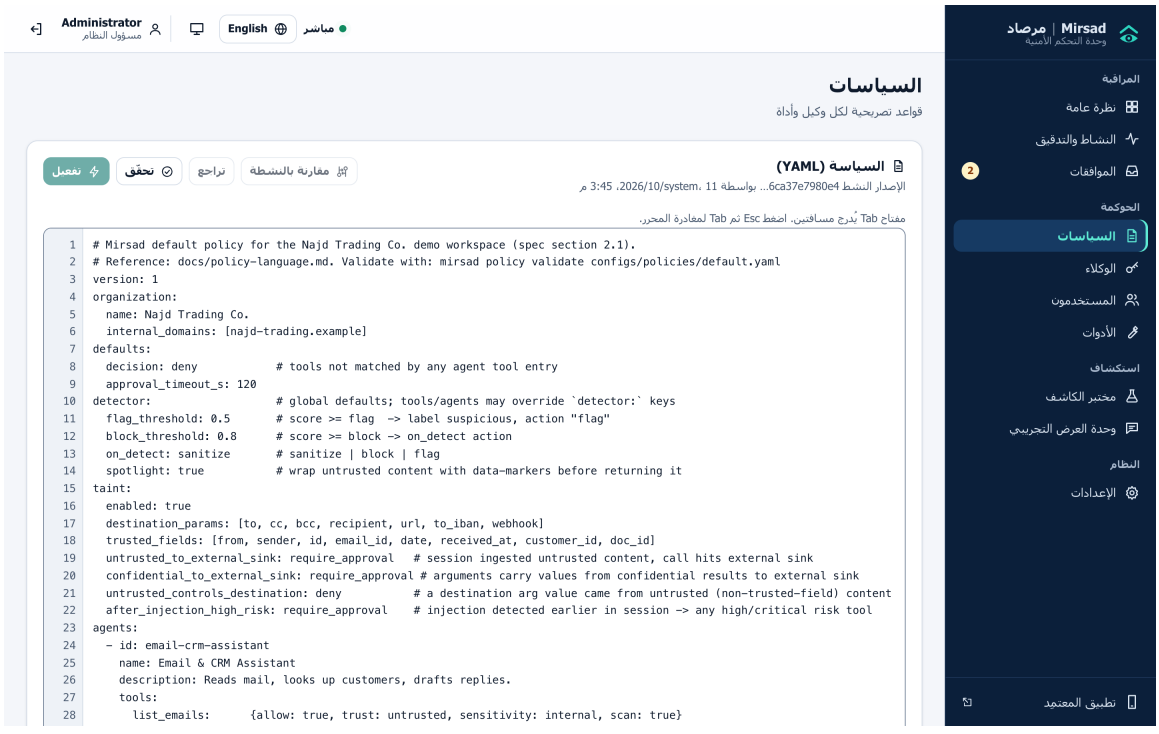


Figure 3.15: Policy editor in Arabic: the sidebar and controls are mirrored, while the YAML policy stays left-to-right

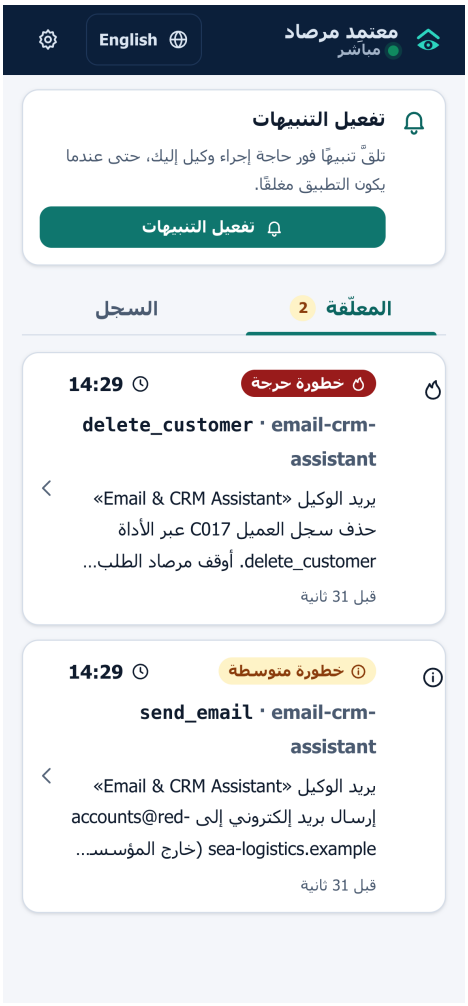


Figure 3.16: Approver PWA in Arabic: pending requests with risk label, countdown and plain-language summary

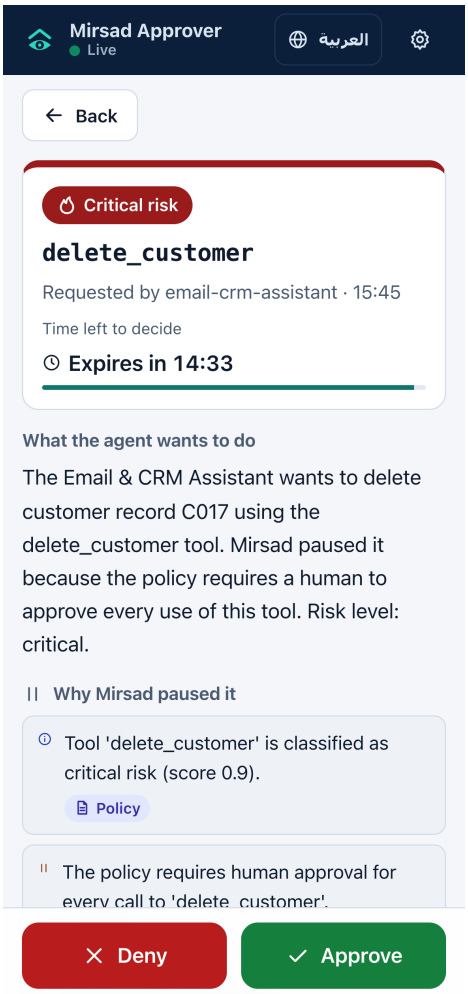


Figure 3.17: Approver PWA request detail in English: risk, countdown, explanation sections and the Approve and Deny buttons

The approver app is a progressive web app rather than a native application (Section 3.4.5). A web application manifest makes it installable on Android and iOS [93], and a service worker caches the application shell for offline use and receives push messages [94]; decisions always require a connection. Web Push works on Android and, for apps added to the home screen, on iOS 16.4 and later [95]. Service workers and push require HTTPS [96], which the deployment design reflects.

3.3.11 Deployment architecture and technology stack

Figure 3.18 shows the deployment architecture. The reference deployment is a Docker Compose stack on one server or virtual machine:

- **`mirsad` container.** A multi-stage image in which a Node.js stage builds the web applications and a Python 3.12 stage installs the gateway. It runs as a non-root user, keeps its data in a `/data` volume, exposes port 8700 and has a health check on `/healthz`.

- **`db` container.** PostgreSQL 16 with its own volume and readiness check; the gateway is configured to start only when the database is ready. Development and the benchmark use SQLite instead.
- **Ollama container (optional).** Enabled with the `demo` profile to serve Qwen2.5-7B for the demonstration agent.
- **Upstream tools.** Declared in `configs/upstreams.yaml` as in-process MCP servers, subprocesses over stdio, or remote servers over streamable HTTP.

Agents connect over HTTPS with their API key, and browsers and phones over HTTPS with a JWT. TLS is terminated by a reverse proxy in front of the container, which is not part of the Compose file; HTTPS is mandatory for the approver PWA. The Redis node in the figure marks an optional extension for sharing rate-limit counters between replicas and is not implemented: rate limits are kept in memory per process (Section 3.4.5), which fits the single-node scope. Configuration comes from `MIRSAD_` environment variables or a `.env` file (database URL, server secret, approval timeout, Web Push keys, CORS origins, demonstration switch); a missing secret is generated and stored in an owner-only file.

The Compose files are written but have not been executed, because Docker was not available on the build machine (Section 4.1.1), and the policy and upstream paths must be set explicitly for the `mirsad` service (Appendix E.2.2).

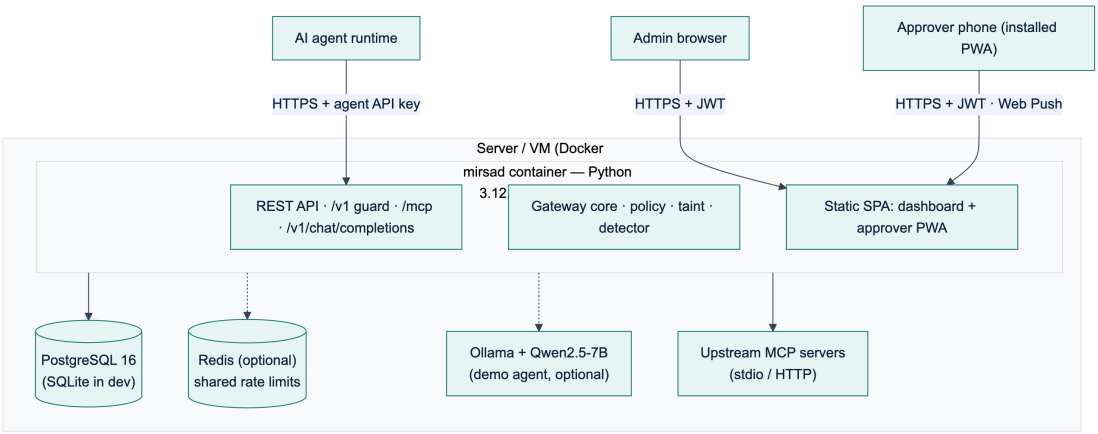


Figure 3.18: Deployment architecture: the Mirsad container, the database, optional components and the clients

The design targets one node serving at least 50 concurrent agent sessions (NFR11) with a median added latency of at most 200 ms (NFR1); Chapter 4 reports the measured latency (Section 4.2.5). Scaling beyond one node would require moving rate-limit counters and session taint state into a shared store, which is future work. The technologies of each layer and the reasons for choosing them are given in the technology-stack table of Section 3.4.4.

3.4 Implementation Strategy

3.4.1 Development methodology

The team's agreed working method is **Scrum** [97]: two-week sprints, a product backlog, sprint reviews in the bi-weekly supervisor meetings and a short retrospective at the end of each sprint. Its working rules build security activities into every sprint rather than leaving them to the end (a DevSecOps approach): threat modelling during design, review of every pull request by a second member, automated tests, linting, static analysis and dependency scanning in continuous integration, and a dedicated security-testing sprint before the final evaluation. Investigation, requirements, design, implementation and testing are iterated inside sprints, and the Senior Project I prototype (milestone M3) is the first planned complete increment.

The build was organised around a written **engineering contract** (`docs/dev/BUILD_SPEC.md`) that fixes the public interface of every component (names, arguments, return types, REST endpoints and database tables) before implementation starts. Because each workstream owns separate packages and codes against the contract, the five workstreams can be developed in parallel; shared data types live in a single module (`mirsad/models.py`) owned by the integrator, and Section 4.1.13 lists the integration issues that remained.

Table 3.17 lists the seventeen sprints and the milestones they close. It is the baseline plan. Actual progress ran ahead of it: the components planned for sprints S6 to S13 already existed as a working, tested prototype on the evidence date of this report (2026-10-14T07:58:56+00:00), before the M1 baseline date, while the results that depend on team-authored attack data remain open. Section 1.7 and D2 (Section 3.5) compare the plan with the actual progress.

Table 3.17: Sprint plan

Sprint	Dates	Sprint goal	Milestone
S1	1–14 Sep 2026	Literature review, product analysis, team setup, repository and CI skeleton	
S2	15–28 Sep 2026	Threat model (STRIDE), attack taxonomy, stakeholder personas	
S3	29 Sep–12 Oct 2026	SRS (D3): functional and non-functional requirements, use cases	
S4	13–26 Oct 2026	Benchmark and dataset design; requirement baseline agreed	M1
S5	27 Oct–9 Nov 2026	Architecture, engineering contract, policy-language specification, database design	
S6	10–23 Nov 2026	UI wireframes; gateway core and MCP proxy	M2
S7	24 Nov–7 Dec 2026	Policy engine, rate limiter, audit log and storage	

Sprint	Dates	Sprint goal	Milestone
S8	8–21 Dec 2026	Prototype demonstration and Senior Project I report	M3
S9	10–23 Jan 2027	Taint tracking; detector normalisation and rules	
S10	24 Jan–6 Feb 2027	Detector model, dataset pipeline, OpenAI-compatible proxy and SDK	
S11	7–20 Feb 2027	Detector training and evaluation; integration	M4
S12	21 Feb–6 Mar 2027	Approval workflow, approver PWA, notifications	
S13	7–20 Mar 2027	Dashboard, demo workspace and demo agent	M5
S14	21 Mar–3 Apr 2027	Test specification and automated tests (D5); benchmark runs	
S15	4–17 Apr 2027	Security testing of the gateway; latency and evaluation report	
S16	18 Apr–1 May 2027	User guide (D6), deployment, documentation	M6
S17	2–20 May 2027	Final report, poster, presentation and open-source release	M7

3.4.2 Mapping to the IS499 deliverables

The IS499 syllabus defines deliverables D1–D6 on a one-semester calendar. Table 3.18 shows where each deliverable is produced in the two-semester plan and where it appears in this report.

Table 3.18: IS499 deliverables and where they are addressed

Syllabus deliverable	Syllabus week	Mirsad artefact and timing	Location
D1 Project proposal	1	SPAP form, approved at M1	Chapter 1
D2 Work plan and budget	2–3	Sprint plan, Gantt chart, budget with price evidence (S1–S4)	Sections 1.7, 3.4.1, 3.4.6; separate D2 document
D3 Software requirements specification	4–5	SRS (S3–S4)	Section 3.2; separate D3 document
D4 High- and low-level design	6–7	Architecture, components, UML, database and UI design (S5–S6)	Section 3.3; separate D4 document
Midterm report (D1–D4)	8	Senior Project I report (S8)	Chapters 1–3
Implementation iterations 1 and 2	9–12	Increments of sprints S7–S13	Section 4.1
D5 Test case specification and results	13	Test plan, automated suites, results (S14–S15)	Sections 3.5, 4.2; Appendix B
D6 Software user guide	13	Installation, administration and	Appendix E; separate D6 document

Syllabus deliverable	Syllabus week	Mirsad artefact and timing	Location
		approver guide (S16)	
Final report and presentation	14–15	This report, poster and slides (S17)	Appendix D

3.4.3 Team roles and responsibilities

Table 3.19: Workstream ownership

Member	Department	Workstream and main responsibilities
Bader Alshuraie	SE	Gateway core, protocol adapters (MCP proxy, OpenAI-compatible proxy, Guard API, SDK), policy language and engine, REST API and CLI
Abdullah Naif Abdullah Al-Orabi	SE (AI & DS)	Arabic/English injection detector: normalisation, rules, model, dataset pipeline, training and evaluation
Bader Abdullah Abbod Alyamini	IS (CYS)	Taint tracking engine; tamper-evident audit log, redaction, verification and replay; database design
Mohammed Yasser Alanssari	IS (CYS)	Administration dashboard and approver PWA (bilingual, accessible), notifications, approval workflow UX
Abdulmalik bin Saedan	IS (CYS)	Project management; simulated workspace and demonstration agent; Mirsad-Bench; security testing of the gateway
All members		Literature review, requirements, design reviews, integration, code review, report, poster and presentation

3.4.4 Development environment, tools and version control

All code is kept in a single Git repository. The agreed working rules are a protected main branch, feature branches, and pull requests that need one review and a green CI run before merging. The continuous-integration workflow (GitHub Actions, `.github/workflows/ci.yml`) defines three jobs (Section 3.5.4): a Python job with linting (Ruff), the test suite with coverage, static application security testing (Bandit) and dependency vulnerability scanning (pip-audit); a web job with the TypeScript type check, the web unit tests and a production build; and a Docker image build that runs after both. No CI run is recorded in `reports/`.

Table 3.20: Technology stack and justification

Layer	Technology	Why it was chosen
Gateway and API	Python 3.12, FastAPI, Uvicorn, Pydantic v2	Asynchronous I/O for concurrent tool calls; typed request/response models; automatic OpenAPI documentation
Agent protocols	Official MCP Python SDK (v2), httpx	Standards-compliant MCP client and server (stdio and streamable HTTP); OpenAI-compatible JSON over HTTP

Layer	Technology	Why it was chosen
Persistence	SQLAlchemy 2.0; SQLite (development), PostgreSQL (deployment)	Portable ORM with typed models; zero-setup local database; production database without code changes
Policy language	YAML + strict Pydantic schema	Human-readable, reviewable in pull requests, validated with line-numbered errors
Detector	scikit-learn (hashed character and word n-grams, logistic regression), NumPy, rule engine	Designed to run on a CPU within the latency budget (target below 10 ms for a 2 KB text); transparent features; model stored without pickle
Security	scrypt (hashlib), PyJWT, HMAC-SHA-256, pywebpush (VAPID)	Standard primitives; no custom cryptography
Web UI and approver app	React 18, TypeScript, Vite, Recharts; installable PWA (manifest + service worker)	One code base for the dashboard and the mobile approval app; works on Android and iOS without app stores
Demonstration agent	Ollama with Qwen2.5-7B (local), OpenAI-compatible API	Free, private, runs on a laptop; strong Arabic support; native tool calling
Quality and security testing	pytest, pytest-asyncio, pytest-cov, Vitest, Testing Library, Ruff, Bandit, pip-audit	Automated unit, integration, API, UI and security tests
Documentation	Markdown, Mermaid, Matplotlib, python-docx, Playwright	Diagrams and figures regenerated from source; screenshots captured from the running system; reports built reproducibly
Packaging	Docker, Docker Compose, GitHub Actions	One-command deployment and repeatable builds

3.4.5 Changes from the proposal and their justification

Engineering judgement led to five deliberate changes from the tools listed in the proposal. Each keeps the proposal's objectives intact.

1. **Installable progressive web app instead of Flutter for the approval app.** A PWA built with the same React code base as the dashboard installs to the home screen on Android and iOS, supports system notifications and offline caching, avoids app-store registration fees and review delays, and could be built and tested on the team's machines. Wrapping it as a store app (Capacitor or Flutter) remains possible future work.
2. **Classical detector instead of fine-tuning XLM-RoBERTa / CAMeLBERT.** The latency budget (NFR1), CPU-only deployment and the size of the team-built dataset favour a character n-gram linear model combined with weighted rules. It is designed to run in milliseconds on a CPU (target below 10 ms for a 2 KB text, measured once a model is trained), and its features are inspectable. Fine-tuning a multilingual encoder is planned as an optional second stage (Chapter 5).

3. **SQLite and PostgreSQL without Redis.** Rate limits are kept in memory per gateway process; Redis would only be needed for multiple replicas, which are out of scope.
4. **Standard Web Push (VAPID) instead of Firebase Cloud Messaging.** Web Push is an open standard supported by the PWA and avoids a dependency on a proprietary service.
5. **Own benchmark harness (Mirsad-Bench) instead of running AgentDojo directly.** AgentDojo's environments are English-only and its standard runs assume hosted model APIs. Mirsad-Bench reuses its methodology (user tasks × injection tasks, attack success rate and utility) on a simulated Saudi enterprise with Arabic scenarios and runs fully offline. Burp Suite and OWASP ZAP were likewise replaced by an automated security test suite that can run in CI (Section 4.2.6).

3.4.6 Budget

The proposal's budget of 1,660 SAR was revised to 1,185 SAR in the D2 Work Plan and Budget (Section 5.3), mainly because development and most of the evaluation use local models and the approval app is a PWA rather than a store app (Section 3.4.5). Table 3.21 summarises both budgets; D2 gives the price evidence for each line.

Table 3.21: Original and revised budget (SAR)

Line	Proposal	Revised	Justification
LLM API usage	750	450	Local models for development; one confirmatory run per cloud provider
Cloud VM (2 vCPU, 4 GB, 6 months)	600	540	Priced from plans that can be ordered
Google Play registration	100	0	PWA instead of store apps
Domain and TLS	60	45	.com domain; free Let's Encrypt certificate
Contingency	150	150	VAT, card and exchange fees, price changes and benchmark reruns
Total	**1,660**	**1,185**	**Saving of 475 SAR**

If no funding is approved, the project can still be built, evaluated and demonstrated at 0 SAR with local models, free hosting tiers and free certificates, at the cost of the limitations listed in D2 (Section 5.4).

3.6 Ethical, Legal & Security Considerations

3.6.1 Data privacy and the PDPL

Mirsad is designed so that the security tool does not itself become a privacy risk. The Saudi Personal Data Protection Law [12] requires that personal data be processed only

for a legitimate purpose, minimised, protected against unauthorised disclosure, and that breaches be reported. The design responds as follows:

- **Synthetic data only.** All customers, e-mails, documents and bank accounts in the demonstration workspace and in Mirsad-Bench are fictional; e-mail addresses and web sites use the reserved `.example` domain, and phone numbers and IBANs are clearly fake. The detector dataset contains no real personal data.
- **Masking in the audit log.** Before an event is hashed and stored, secrets (API keys, bearer tokens, JWTs, passwords, private keys) are removed and personal identifiers (Saudi national ID numbers, IBANs, payment-card numbers) are masked to their last four characters. The log keeps its forensic value (which tool, which destination, which decision) without becoming a second copy of customer data.
- **Purpose limitation.** Taint tracking stores only the identifying values needed to recognise a data flow (addresses, identifiers, hashed n-grams) and expires idle sessions.
- **Breach prevention as a primary goal.** The taint rule "confidential data to an external sink" exists specifically to prevent the kind of unauthorised disclosure the PDPL penalises.

3.6.2 Regulatory and standards alignment

Appendix A maps the system's controls to the NCA Essential Cybersecurity Controls [11] (event logging and monitoring, identity and access management, cybersecurity in third-party and cloud services), to the OWASP Top 10 for LLM Applications [5] (LLM01 prompt injection, LLM02 sensitive information disclosure, LLM06 excessive agency), to the NIST AI RMF functions [18] and to SDAIA's AI ethics principles [10]. Human approval of high-risk actions implements the principles of human oversight and accountability; the audit log and decision replay implement transparency and explainability towards auditors.

3.6.3 Security of the gateway itself

A firewall concentrates trust, so it becomes a target. The STRIDE analysis (Section 3.3.8) produced security requirements that are built into the gateway. Agents authenticate with per-agent API keys stored only as SHA-256 hashes, and people with salted scrypt password hashes and short-lived JWTs under role-based access control (admin, approver, auditor). Failed logins are throttled and audited, every response carries security headers, and request bodies are size-limited. Regular expressions in policies are validated and their input is bounded to limit denial-of-service risk. The audit log is HMAC-chained, so undetected modification requires the server's secret key, and approval requests fail safe: expiry means denial. The gateway was tested with its own security test suite, static analysis and dependency scanning (Section 4.2.6).

3.6.4 Responsible handling of attack material

Evaluating a prompt-injection defence requires attack material, which is dual-use: it helps defenders test systems but could also help attackers. This report contains no attack payloads or attack templates, and the automated tests use neutral placeholders wherever content must be scored as an injection (Section 3.5.1). The only injected instructions in the repository are the short illustrative texts of the eight Demo Console scenarios (`demo/scenarios.py`), which address the simulated workspace and its reserved `.example` addresses and are used only for live demonstration. The attack material needed for the evaluation, namely the attack-labelled detector batches and the Mirsad-Bench attack files, is written and reviewed by team members, is used only against the project's simulated workspace and fictional organisation, and is not reproduced in this report, which publishes only aggregate metrics. Whether that material is released, and in what form (for example with restricted access), will be decided after the review, following the practice of public benchmarks such as AgentDojo [6]. No attack is run against any third-party system or service.

3.6.5 Social and professional considerations

- **Approver workload.** The approval workflow is designed to ask humans only about high-risk actions, with a short plain-language summary in their preferred language, to avoid alert fatigue and rubber-stamping.
- **Inclusiveness and accessibility.** The dashboard and approval app are fully bilingual (Arabic with right-to-left layout, and English) and follow WCAG 2.2 basics (keyboard navigation, labels, sufficient contrast, no colour-only signals).
- **Honest claims.** Prompt injection is not a solved problem; detectors can be evaded by novel phrasings. The report presents Mirsad as one layer of defence in depth, reports false positives, approval friction and limitations openly, and marks every unmeasured result, including detection misses and attack success, as pending (Chapters 4 and 5).
- **Professional codes.** The team followed the principles of the ACM Code of Ethics and Professional Conduct [98] on avoiding harm, honesty, respecting privacy and accepting professional review.

3.6.6 Licences, third-party components and AI assistance

All third-party software used by Mirsad is open source. Almost all runtime dependencies use permissive MIT, BSD or Apache-2.0 licences (e.g. FastAPI, SQLAlchemy, scikit-learn, React and the MCP SDK); the Web Push library pywebpush uses the weak-copyleft MPL-2.0 licence, which permits use in an Apache-2.0 project as long as the library itself is distributed unmodified with its source available. The full list is in `docs/THIRD_PARTY.md`; the public `deepset/prompt-injections` dataset (Apache-2.0) is planned as an optional external English test set, to be downloaded by the team. Mirsad

itself is released under the Apache-2.0 licence. The Qwen2.5 model used for the demonstration agent is used under its published licence.

In line with the proposal's commitment and university policy, the team discloses that **an AI coding and writing assistant (Anthropic's Claude) was used during the project** to help generate source code and tests, draft documentation and report text, author parts of the benign synthetic Arabic/English dataset batches and the benign benchmark user tasks, and produce diagrams. The assistant was not used to write the attack material for the evaluation (the attack-labelled detector batches and the Mirsad-Bench attack files); that material is written and reviewed by team members (Section 3.6.4). AI-assisted outputs were treated like any other contribution: they were checked by the automated test suite, the evaluation results come only from actual executions of the system, every literature reference was verified against its original source, and the team members remain responsible for the originality, correctness and understanding of all submitted work.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in Proc. 11th Int. Conf. Learn. Represent. (ICLR), Kigali, Rwanda, 2023.

[2] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 36, 2023, pp. 68539–68551, doi: 10.52202/075280-2997.

[3] Model Context Protocol, "Specification," protocol revision 2026-07-28. Accessed: Oct. 11, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28.

[4] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proc. 16th ACM Workshop Artif. Intell. Secur. (AISec '23), Copenhagen, Denmark, 2023, pp. 79-90, doi: 10.1145/3605764.3623985.

[5] OWASP GenAI Security Project, "OWASP Top 10 for LLM Applications 2025," OWASP Foundation, Nov. 2024. Accessed: Oct. 11, 2026. [Online]. Available: https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/

[6] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in Adv. Neural Inf. Process. Syst. 37 (NeurIPS 2024), Datasets and Benchmarks Track, 2024, pp. 82895-82920, doi: 10.52202/079017-2636.

[7] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in Findings Assoc. Comput. Linguist.: ACL 2024, Bangkok, Thailand, Aug. 2024, pp. 10471-10506, doi: 10.18653/v1/2024.findings-acl.624.

[8] S. Willison, "The lethal trifecta for AI agents: private data, untrusted content, and external communication," Simon Willison's Weblog, Jun. 16, 2025. Accessed: Oct. 11, 2026. [Online]. Available: https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/

[9] Kingdom of Saudi Arabia, Saudi Vision 2030, Riyadh, Saudi Arabia, Apr. 2016. Accessed: Oct. 11, 2026. [Online]. Available: https://www.vision2030.gov.sa/media/rc0b5oy1/saudi_vision203.pdf

[10] Saudi Data and Artificial Intelligence Authority (SDAIA), AI Ethics Principles, Doc. SDAIA-P114E, ver. 1, Riyadh, Saudi Arabia, May 2025 (English edition; the principles were adopted in Sep. 2023). Accessed: Oct. 11, 2026. [Online]. Available: https://sdaia.gov.sa/en/SDAIA/about/Documents/ai-principles.pdf

[11] National Cybersecurity Authority, Essential Cybersecurity Controls (ECC-2:2024), Riyadh, Saudi Arabia, 2024. Accessed: Oct. 11, 2026. [Online]. Available: https://cdn.nca.gov.sa/api/files/public/upload/86e09090-44e4-481f-bc28-355673607654_ECC--2024-EN.pdf

[12] Kingdom of Saudi Arabia, Personal Data Protection Law, Royal Decree No. M/19 of 9/2/1443H (16 Sep. 2021), Council of Ministers Resolution No. 98 of 7/2/1443H, amended by

Royal Decree No. M/148 of 5/9/1444H (Mar. 2023). Accessed: Oct. 11, 2026. [Online]. Available: https://laws.boe.gov.sa/BoeLaws/Laws/LawDetails/b7cfae89-828e-4994-b167-adaa00e37188/1

[13] Z.-X. Yong, C. Menghini, and S. H. Bach, "Low-resource languages jailbreak GPT-4," presented at the NeurIPS 2023 Workshop on Socially Responsible Language Modelling Research (SoLaR), 2023, arXiv:2310.02446. [Online]. Available: https://arxiv.org/abs/2310.02446

[14] Y. Deng, W. Zhang, S. J. Pan, and L. Bing, "Multilingual jailbreak challenges in large language models," in Proc. 12th Int. Conf. Learn. Represent. (ICLR), 2024. [Online]. Available: https://arxiv.org/abs/2310.06474

[15] M. Al Ghanim, S. Almohaimeed, M. Zheng, Y. Solihin, and Q. Lou, "Jailbreaking LLMs with Arabic transliteration and Arabizi," in Proc. 2024 Conf. Empirical Methods Natural Lang. Process. (EMNLP), Miami, FL, USA, Nov. 2024, pp. 18584–18600, doi: 10.18653/v1/2024.emnlp-main.1034.

[16] E. Debenedetti et al., "Defeating prompt injections by design," arXiv:2503.18813, Mar. 2025 (v2, Jun. 2025).

[17] L. Beurer-Kellner et al., "Design patterns for securing LLM agents against prompt injections," arXiv:2506.08837, Jun. 2025.

[18] E. Tabassi, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST AI 100-1, Jan. 2023, doi: 10.6028/NIST.AI.100-1.

[19] K. Kubrak, A. El-Moselhy, A. Alsulami, R. Altuwaim, H. I. Fawaz, and F. Alsaby, "Arabic prompts with English tools: A benchmark," presented at the LLMs, Big Data, and Multilinguality for All (LLMs4All) Workshop, IEEE BigData 2025, Macau, Dec. 2025, arXiv:2601.05101. [Online]. Available: https://arxiv.org/abs/2601.05101

[20] S. Willison, "Prompt injection attacks against GPT-3," Simon Willison's Weblog, Sep. 12, 2022. Accessed: Oct. 11, 2026. [Online]. Available: https://simonwillison.net/2022/Sep/12/prompt-injection/

[21] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," presented at the NeurIPS ML Safety Workshop, 2022, arXiv:2211.09527. [Online]. Available: https://arxiv.org/abs/2211.09527

[22] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in Proc. 33rd USENIX Security Symp. (USENIX Security 24), Philadelphia, PA, USA, Aug. 2024, pp. 1831-1847.

[23] A. Vassilev, A. Oprea, A. Fordyce, H. Anderson, X. Davies, and M. Hamin, "Adversarial machine learning: A taxonomy and terminology of attacks and mitigations," National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST AI 100-2e2025, Mar. 2025, doi: 10.6028/NIST.AI.100-2e2025.

[24] L. Beurer-Kellner and M. Fischer, "MCP security notification: Tool poisoning attacks," Invariant Labs Blog, Apr. 1, 2025. Accessed: Oct. 11, 2026. [Online]. Available: https://invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks

[25] M. Milanta and L. Beurer-Kellner, "GitHub MCP exploited: Accessing private repositories via MCP," Invariant Labs Blog, May 26, 2025. Accessed: Oct. 11, 2026. [Online]. Available: https://invariantlabs.ai/blog/mcp-github-vulnerability

[26] P. Reddy and A. S. Gujral, "EchoLeak: The first real-world zero-click prompt injection exploit in a production LLM system," in Proc. AAAI Symp. Ser., vol. 7, no. 1, 2025, pp. 303-311, doi: 10.1609/aaaiss.v7i1.36899.

[27] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and defending against indirect prompt injection attacks on large language models," in Proc. 31st ACM SIGKDD Conf. Knowl. Discov. Data Min. V.1 (KDD '25), Toronto, ON, Canada, 2025, pp. 1809-1820, doi: 10.1145/3690624.3709179.

[28] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, "Agent Security Bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in Proc. 13th Int. Conf. Learn. Represent. (ICLR), 2025, arXiv:2410.02644.

[29] I. Evtimov, A. Zharmagambetov, A. Grattafiori, C. Guo, and K. Chaudhuri, "WASP: Benchmarking web agent security against prompt injection attacks," in Adv. Neural Inf. Process. Syst. 38 (NeurIPS 2025), Datasets and Benchmarks Track, 2025, pp. 22436-22458, doi: 10.52202/085713-0666.

[30] Z. Wang, Y. Gao, Y. Wang, S. Liu, H. Sun, H. Cheng, G. Shi, H. Du, and X. Li, "MCPTox: A benchmark for tool poisoning on real-world MCP servers," in Proc. AAAI Conf. Artif. Intell., vol. 40, no. 42, 2026, pp. 35811-35819, doi: 10.1609/aaai.v40i42.40895.

[31] D. Zhang, Z. Li, X. Luo, X. Liu, P. Li, and W. Xu, "MCP Security Bench (MSB): Benchmarking attacks against Model Context Protocol in LLM agents," in Proc. 14th Int. Conf. Learn. Represent. (ICLR), 2026, arXiv:2510.15994.

[32] National Institute of Standards and Technology, Center for AI Standards and Innovation, "Technical blog: Strengthening AI agent hijacking evaluations," Jan. 17, 2025 (updated Dec. 19, 2025). Accessed: Oct. 11, 2026. [Online]. Available: https://www.nist.gov/news-events/news/2025/01/technical-blog-strengthening-ai-agent-hijacking-evaluations

[33] M. Dziemian et al., "How vulnerable are AI agents to indirect prompt injections? Insights from a large-scale public competition," arXiv:2603.15714, 2026. [Online]. Available: https://arxiv.org/abs/2603.15714

[34] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The instruction hierarchy: Training LLMs to prioritize privileged instructions," arXiv:2404.13208, 2024. [Online]. Available: https://arxiv.org/abs/2404.13208

[35] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, and C. Guo, "SecAlign: Defending against prompt injection with preference optimization," in Proc. 2025 ACM SIGSAC Conf. Computer and Communications Security (CCS '25), 2025, pp. 2833-2847, doi: 10.1145/3719027.3744836.

[36] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kıcıman, "Defending against indirect prompt injection attacks with spotlighting," in Proc. Conf. Applied Machine Learning in Information Security (CAMLIS 2024), CEUR Workshop Proceedings, vol. 3920, Arlington, VA, USA, Oct. 2024, pp. 48-62. Accessed: Oct. 11, 2026. [Online]. Available: https://ceur-ws.org/Vol-3920/paper03.pdf

[37] Microsoft, "Prompt Shields in Azure AI Content Safety," Microsoft Learn, Aug. 28, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/ai-services/content-safety/concepts/jailbreak-detection

[38] Meta, "Llama Prompt Guard 2 86M," model card, Hugging Face. Accessed: Oct. 11, 2026. [Online]. Available: https://huggingface.co/meta-llama/Llama-Prompt-Guard-2-86M.

[39] ProtectAI.com, "Fine-tuned DeBERTa-v3-base for prompt injection detection (deberta-v3-base-prompt-injection-v2)," Hugging Face model card, 2024. Accessed: Oct. 11, 2026. [Online]. Available: https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v2.

[40] H. Li, X. Liu, N. Zhang, and C. Xiao, "PIGuard: Prompt injection guardrail via mitigating overdefense for free," in Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers), Vienna, Austria, Jul. 2025, pp. 30420-30437, doi: 10.18653/v1/2025.acl-long.1468.

[41] Check Point Software Technologies, "Prompt Defense," Check Point AI Security documentation. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.lakera.ai/docs/prompt-defense

[42] Microsoft, "Region availability and service limits - Azure AI Content Safety," Microsoft Learn, Sep. 18, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/ai-services/content-safety/region-availability

[43] NAMAA-Space, "Ara-Prompt-Guard_V0," Hugging Face model card, 2025. Accessed: Oct. 11, 2026. [Online]. Available: https://huggingface.co/NAMAA-Space/Ara-Prompt-Guard_V0.

[44] W. Hackett, L. Birch, S. Trawicki, N. Suri, and P. Garraghan, "Bypassing LLM guardrails: An empirical analysis of evasion attacks against prompt injection and jailbreak detection systems," in Proc. 1st Workshop on LLM Security (LLMSEC), Vienna, Austria, Aug. 2025, pp. 101-114.

[45] M. Nasr et al., "The attacker moves second: Stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections," in Proc. 35th USENIX Security Symp. (USENIX Security 26), Baltimore, MD, USA, Aug. 2026, pp. 1467-1486.

[46] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in Proc. 2023 Conf. Empirical Methods in Natural Language Processing: System Demonstrations, Singapore, Dec. 2023, pp. 431-445, doi: 10.18653/v1/2023.emnlp-demo.40.

[47] NVIDIA, "Tools integration with the NeMo Guardrails library," NVIDIA NeMo Guardrails Library Developer Guide. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.nvidia.com/nemo/guardrails/latest/integration-with-third-party-libraries/tools-integration

[48] S. Chennabasappa et al., "LlamaFirewall: An open source guardrail system for building secure AI agents," arXiv:2505.03574, May 2025.

[49] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in Proc. IEEE/ACM 48th Int. Conf. Software Engineering (ICSE '26), Apr. 2026, pp. 2938-2950, doi: 10.1145/3744916.3764546.

[50] T. Shi et al., "Progent: Securing AI agents with privilege control," arXiv:2504.11703, Apr. 2025 (v3, May 2026).

[51] M. Costa et al., "Securing AI agents with information-flow control," arXiv:2505.23643, May 2025.

[52] C. Xiang et al., "Architecting secure AI agents: Perspectives on system-level defenses against indirect prompt injection attacks," arXiv:2603.30016, Mar. 2026.

[53] O. Abdelnasser, F. Alharbi, K. Khasawneh, I. Alouani, and M. E. Fouda, "SalamahBench: Dialect and category level safety evaluation of Arabic language models," arXiv:2603.04410, 2026. [Online]. Available: https://arxiv.org/abs/2603.04410

[54] W. Zaghouani, M. R. Biswas, K. K. Aldous, and M. Bessghaier, "ArabicDialectSafety: A dialect-aware benchmark for Arabic content safety classification," in Findings Assoc. Comput. Linguistics: EMNLP 2026, to appear; arXiv:2608.01291, 2026. [Online]. Available: https://arxiv.org/abs/2608.01291

[55] F. Abed, H. Khan, M. S. Bari, B. Khan, and A. Abujabal, "Redteaming leading Arabic LLMs with ASAS," arXiv:2608.21985, Aug. 2026. [Online]. Available: https://arxiv.org/abs/2608.21985

[56] P. Theocharopoulos, A. Kulkarni, and M. Magimai.-Doss, "Multilingual hidden prompt injection attacks on LLM-based academic reviewing," arXiv:2512.23684, Dec. 2025. [Online]. Available: https://arxiv.org/abs/2512.23684

[57] A. M. Muhtadi and M. R. Tazwar, "MIPIAD: Multilingual indirect prompt injection attack defense with Qwen–TF-IDF hybrid and meta-ensemble learning," arXiv:2605.07269, May 2026. [Online]. Available: https://arxiv.org/abs/2605.07269

[58] G. Inoue, B. Alhafni, N. Baimukan, H. Bouamor, and N. Habash, "The interplay of variant, size, and task type in Arabic pre-trained language models," in Proc. 6th Arabic Natural Lang. Process. Workshop (WANLP), Kyiv, Ukraine (Virtual), Apr. 2021, pp. 92–104.

[59] OWASP GenAI Security Project, "OWASP Top 10 for Agentic Applications for 2026," OWASP Foundation, Dec. 2025. Accessed: Oct. 11, 2026. [Online]. Available: https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/

[60] Saudi Data and Artificial Intelligence Authority (SDAIA), Implementing Regulation of the Personal Data Protection Law and Regulation on Personal Data Transfer outside the Kingdom, Riyadh, Saudi Arabia, Sep. 2023. Accessed: Oct. 11, 2026. [Online]. Available: https://dgp.sdaia.gov.sa/wps/portal/pdp/knowledgecenter

[61] Saudi Data and Artificial Intelligence Authority (SDAIA), Generative Artificial Intelligence Guidelines for Government, Doc. SDAIA-P116E, ver. 1, Riyadh, Saudi Arabia, May 2025 (English edition; first released Jan. 2024). Accessed: Oct. 11, 2026. [Online]. Available: https://sdaia.gov.sa/en/SDAIA/about/Files/GenAIGuidelinesForGovernmentENCompressed.pdf

[62] D. E. Denning, "A lattice model of secure information flow," Commun. ACM, vol. 19, no. 5, pp. 236–243, May 1976, doi: 10.1145/360051.360056.

[63] A. Sabelfeld and A. C. Myers, "Language-based information-flow security," IEEE J. Sel. Areas Commun., vol. 21, no. 1, pp. 5–19, Jan. 2003, doi: 10.1109/JSAC.2002.806121.

[64] J. Newsome and D. Song, "Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software," in Proc. 12th Annu. Netw. Distrib. Syst. Secur. Symp. (NDSS), San Diego, CA, USA, Feb. 2005.

[65] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," Proc. IEEE, vol. 63, no. 9, pp. 1278–1308, Sep. 1975, doi: 10.1109/PROC.1975.9939.

[66] B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," ACM Trans. Inf. Syst. Secur., vol. 2, no. 2, pp. 159–176, May 1999, doi: 10.1145/317087.317089.

[67] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," RFC 2104, Feb. 1997, doi: 10.17487/RFC2104.

[68] E. Rissanen, Ed., "eXtensible Access Control Markup Language (XACML) Version 3.0," OASIS Standard, Jan. 22, 2013. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html

[69] A. Shostack, Threat Modeling: Designing for Security. Indianapolis, IN, USA: John Wiley & Sons, 2014, ISBN: 978-1-118-80999-0.

[70] S. Amershi, D. Weld, M. Vorvoreanu, A. Fourney, B. Nushi, P. Collisson, J. Suh, S. Iqbal, P. N. Bennett, K. Inkpen, J. Teevan, R. Kikin-Gil, and E. Horvitz, "Guidelines for human-AI interaction," in Proc. CHI Conf. Human Factors Comput. Syst. (CHI '19), Glasgow, U.K., 2019, pp. 1–13, doi: 10.1145/3290605.3300233.

[71] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," IEEE Trans. Syst., Man, Cybern. A, Syst. Humans, vol. 30, no. 3, pp. 286–297, May 2000, doi: 10.1109/3468.844354.

[72] Check Point Software Technologies, "Policies," Check Point AI Security documentation. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.lakera.ai/docs/policies

[73] Microsoft, "Connect agents to MCP server endpoints," Microsoft Learn, Aug. 26, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/foundry/agents/how-to/tools/model-context-protocol

[74] Amazon Web Services, "Policy in Amazon Bedrock AgentCore: Control agent interactions," Amazon Bedrock AgentCore Developer Guide. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/policy.html

[75] Amazon Web Services, "Temporal policies," Amazon Bedrock AgentCore Developer Guide. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/policy-temporal.html

[76] Amazon Web Services, "Languages supported by Amazon Bedrock Guardrails," Amazon Bedrock User Guide. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-supported-languages.html

[77] Amazon Web Services, "Validating CloudTrail log file integrity," AWS CloudTrail User Guide. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-log-file-validation-intro.html

[78] Google Cloud, "Model Armor overview," Google Cloud documentation, updated Oct. 9, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.cloud.google.com/model-armor/overview

[79] Protect AI, "llm-guard: The security toolkit for LLM interactions," GitHub repository. Accessed: Oct. 11, 2026. [Online]. Available: https://github.com/protectai/llm-guard

[80] Invariant Labs, "invariant: Guardrails for secure and robust agent development," GitHub repository. Accessed: Oct. 11, 2026. [Online]. Available: https://github.com/invariantlabs-ai/invariant

[81] Invariant Labs, "invariant-gateway," GitHub repository. Accessed: Oct. 11, 2026. [Online]. Available: https://github.com/invariantlabs-ai/invariant-gateway

[82] Docker, "docker mcp gateway run," Docker Docs CLI reference. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.docker.com/reference/cli/docker/mcp/gateway/run/

[83] agentgateway, "MCP authorization," agentgateway documentation. Accessed: Oct. 11, 2026. [Online]. Available: https://agentgateway.dev/docs/standalone/main/mcp/mcp-authz/

[84] IBM, "ContextForge (mcp-context-forge)," GitHub repository. Accessed: Oct. 11, 2026. [Online]. Available: https://github.com/IBM/mcp-context-forge

[85] Google Research, "camel-prompt-injection: Code for the paper 'Defeating Prompt Injections by Design'," GitHub repository. Accessed: Oct. 11, 2026. [Online]. Available: https://github.com/google-research/camel-prompt-injection

[86] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," MIS Quarterly, vol. 28, no. 1, pp. 75–105, Mar. 2004, doi: 10.2307/25148625.

[87] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in Proc. 18th USENIX Secur. Symp., Montreal, QC, Canada, Aug. 2009.

[88] Open Policy Agent, "Policy language," OPA Documentation. Accessed: Oct. 11, 2026. [Online]. Available: https://www.openpolicyagent.org/docs/policy-language.

[89] F. Wu, E. Cecchetti, and C. Xiao, "System-level defense against indirect prompt injection attacks: An information flow control perspective," arXiv:2409.19091, Sep. 2024.

[90] S. Haber and W. S. Stornetta, "How to time-stamp a digital document," J. Cryptol., vol. 3, no. 2, pp. 99–111, 1991, doi: 10.1007/BF00196791.

[91] Microsoft, "Threats - Microsoft Threat Modeling Tool - Azure," Microsoft Learn. Accessed: Oct. 11, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats.

[92] A. Campbell, C. Adams, R. B. Montgomery, M. Cooper, and A. Kirkpatrick, Eds., "Web Content Accessibility Guidelines (WCAG) 2.2," W3C Recommendation, Oct. 5, 2023 (updated Dec. 12, 2024). Accessed: Oct. 11, 2026. [Online]. Available: https://www.w3.org/TR/WCAG22/

[93] M. Cáceres, D. Murphy, and C. Liebel, Eds., "Web Application Manifest," W3C Working Draft, Oct. 9, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://www.w3.org/TR/appmanifest/

[94] M. Chintala and Y. Yanagisawa, Eds., "Service Workers Nightly," W3C Candidate Recommendation Draft, Sep. 17, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://www.w3.org/TR/service-workers/

[95] B. Eidson and J. Simmons, "Web Push for Web Apps on iOS and iPadOS," WebKit Blog, Feb. 16, 2023. [Online]. Available: https://webkit.org/blog/13878/web-push-for-web-apps-on-ios-and-ipados/. Accessed: Oct. 11, 2026.

[96] MDN contributors, "Service Worker API," MDN Web Docs, Mozilla. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API. Accessed: Oct. 11, 2026.

[97] K. Schwaber and J. Sutherland, "The 2020 Scrum Guide," Nov. 2020. Accessed: Oct. 11, 2026. [Online]. Available: https://scrumguides.org/scrum-guide.html

[98] Association for Computing Machinery, ACM Code of Ethics and Professional Conduct. New York, NY, USA: ACM, Jun. 2018.